



Convolutional neural networks optimization using multi-objective particle swarm optimization algorithm [☆]

Armin Rashno ^a, Sadeq Fadaei ^{b,*}

^a Department of Computer Engineering, Faculty of Engineering, Lorestan University, Khorramabad, Iran

^b Department of Electrical Engineering, Faculty of Engineering, Yasouj University, Yasouj, Iran

ARTICLE INFO

Keywords:

Convolutional neural networks
Architecture optimization
Particle swarm optimization
Heuristic

ABSTRACT

This paper presents a heuristic-based convolutional neural network architecture optimization scheme (HeuCNN). First, CNN architectures are encoded as particles in optimization space with two objectives test error and number of parameters. Then, the effectiveness of CNN particles is represented by a heuristic in optimization space. It is mathematically proven and experimentally shown that the proposed heuristic assigns the best values for CNNs with higher performances. Uniform and nonuniform distribution of CNN particles in optimization space are analyzed and behavior of CNNs moving towards higher test error and higher number of parameters are also mathematically proven. CNN architecture search is proposed based on particle swarm optimization (PSO) by new algorithms for particle difference and velocity computation as well as particle update rules. Limitations of HeuCNN are explained and primary insights are presented to address its challenges as future works. Evaluation of HeuCNN on 9 publicly available datasets and comparing with 38 constant and optimized CNN peer competitive models reveal that HeuCNN achieves the best test error and the lowest number of parameters simultaneously. Source codes, data and software associated with this research will be available online after the publication of the paper.

1. Introduction

Many challenges in image classification can be addressed by convolutional neural networks (CNNs). In CNNs, feature maps are extracted automatically and local connectivity between pixels are considered to extract global information with a hierarchical scheme [1]. CNNs include convolution, pooling and fully-connected layers. Convolution layers apply filters to input data and integrate information underneath filters as outputs. Therefore, local information is extracted by convolution layers. Pooling layers are down sample operators and replace neighboring pixels by their average or maximum. Fully-connected layers creates all-to-all connections between neurons to create desired outputs as same as labels in dataset [2].

To design CNNs architectures, type and order of layers, number of filters, dimension of each filter, stride and dimensions in zero-padding are considered which should be handcrafted carefully. This task is non-trivial since knowledge of domain is needed to configure a high-performance CNN architecture. Another option to design CNNs architectures is search methods [3]. In this case,

[☆] Fully documented templates are available in the elsarticle package on CTAN.

* Corresponding author.

E-mail addresses: rashno.a@lu.ac.ir (A. Rashno), s.fadaei@yu.ac.ir (S. Fadaei).

finding the best CNN architecture is modeled as optimization problem and each CNN architecture is considered as a particle (solution) in optimization space [4].

1.1. Related works

Proposed model in this research leads to optimized CNN model for image classification. Therefore, all methods for image classification based on CNNs are reviewed as related works. Pre-defined CNNs, designed by experts, are considered as constant-architectures. CNNs architectures which are found by optimization methods are referred as optimized-architecture CNNs. Optimization models based on swarm intelligence methods such as ant colony optimization (ACO) and particle swarm optimization have been used in many applications. ACO was used in content-based image retrieval [5], image pixel classification [6] and text-independent speaker verification [7]. PSO was applied to solve optimization problems in multi-objective feature selection [8,9], mathematical functions co-evolution [10], and convolution-layer parameters optimization (CPOCNN) [11]. Deep learning methods are also used in many practical fields such as medical image analysis [12]. Since swarm intelligence methods and deep learning models have been very successful in many applications, optimization of CNNs by PSO is the main contribution of this research. In the following sections, related constant and optimized structures for CNNs are reviewed in detail.

1.1.1. Image classification with constant architectures

Lecun introduced the first CNN model for image classification referred as LeNet [13]. Other well-known CNN models are AlexNet [14], VGGNet [15] and GoogleNet [16]. Invariant scattering convolution networks (ScatNet) was presented based on non-linear modulus and averaging operators followed by cascades wavelet transform convolutions [17]. Deep belief networks (DBN) and stacked auto associators (SAA) and variation neural networks (NNet) were introduced as shallow and deep architectures [18]. A deep learning classifier was proposed in [19] based on cascaded network of principal component analysis (PCANet) and blockwise histograms. Two derivations of PCANet were randomly selection of cascaded filters (RandNet) and linear discriminant analysis networks (LDANet). Boltzmann machine was applied for image classification which led to two models point-wise gated Boltzmann machines (PGBM) [20] and transformation-invariant restricted Boltzmann machine (TIRBM) [21]. To save space of the paper, other models for image classification are not explained and just listed as follows: Dropconnect [22], basic auto-encoder (AE) [23], auto-encoder with weight decay regularization (AEwd) [23], contractive auto-encoders (CAE) [23], and constructive divergence (RBM-binary) [23].

1.1.2. Image classification with optimized architectures

Optimization methods can search CNN architectures in optimization space and find an architecture with the best performance. Both auto-encoders and CNNs can be optimized for image classification. Particle swarm optimization (PSO) was applied as optimization method for marginalized stacked denoising auto-encoder (PSO-mSDA) [24]. Flexible convolutional auto-encoder (FCAE) was proposed in [25] based on PSO to search optimal architectures in optimization space. CNN architectures were designed by firefly algorithm (FA-CNN) [26] and growth algorithm (TGA-CNN) [27]. In [28], a variable-length PSO based on network IP addresses (IPPSO) was introduced to find optimal architectures of CNNs. In [29], quantum computing was developed as a binary encoding strategy CNN optimization (BQCNN). Meaningful deep CNNs were formulated in search space by a fast convergence PSO, referred as psoCNN in [30]. In [31] a hyper-parameter optimization scheme was introduced using PSO. In [32], two models enhanced FA CNN (EFA-CNN) and exploitation enhanced TGA CNN (EE-TGA-CNN) were proposed for CNNs parameters optimization based on swarm intelligence meta-heuristics. Architectures and parameters of CNNs were optimized in two separate levels of PSO optimization processes referred as (MPSO-CNN) [33]. In [34], meta-heuristic-driven methods including cuckoo search (CS), bat algorithm (BA), FA, PSO, Caffe and Dropout Caffe were applied to optimize dropout as regularization parameter in CNN. Weight initialization and architecture search were solved by genetic algorithm (GA) and called evoCNN in [35]. Two objectives error rate and number of parameters were formulated in PSO based on decomposition to optimize CNNs architectures (MOPSO/D) [36]. Finally, in [37], IEA-CNN and CEA-CNN were proposed as two versions of evolutionary algorithms (EAs) including independent and committee EAs for CNN hyper-parameters optimization.

1.2. Challenges, motivations and contributions

Challenges of current CNN optimization models, our motivations to address these challenges and the main contributions of the proposed method in this manuscript are all discussed in this section.

Our study reveals that the following issues have not been addressed yet in current CNN optimization models and can be considered as statement of problem.

1. In CNNs, two objectives including classification performance (test error) and complexity of network are very important. A deep network with high number of parameters although can have a high accuracy, it is not possible to be implemented by resources with low computation power and may be involved with over-fitting. On the other hand, light-weight networks with lower accuracy are not useful in real applications. Therefore, both objectives should be considered in network optimization simultaneously. In [29,38,35,28] and [30], ONLY classification test error is optimized and CNNs with high structural complexity are not penalized. Therefore, complex CNNs with high number of layers and parameters are found as solutions.
2. Proposed methods in [31,32,39,34,37,40] and [41], optimize hyper-parameters of CNNs with constant architecture. Optimization of hyper-parameters in CNNs and considering constant architectures (CNNs with the same order and type of layers) do not make

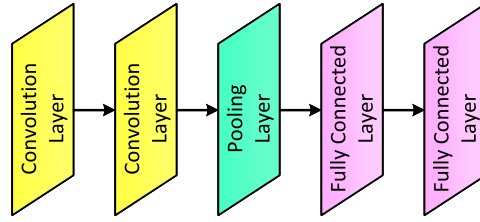


Fig. 1. Integration of layers as a particle in optimization space.

a perfect exploration of optimization space to find high performance CNNs. The reason is that permutation of layers affects CNNs performance significantly which is ignored in these methods.

3. In 2-objective CNN optimization methods [36,33] and [42], CNNs are encoded in optimization space and best solutions are searched. No further information (heuristic) related to effectiveness of CNN particles as well as their behavior regarding each objective are considered in optimization process.

Mentioned drawbacks motivated us to address all current challenges and reach to HeuCNN as a heuristic-based, bi-objective CNN optimization scheme.

The proposed method introduces a heuristic-based CNNs architecture optimization scheme to find CNNs with the best test error and simple structures with the lowest number of parameters (parms). Novelties of this research can be summarized in the following items:

1. CNN architectures are encoded as PSO particles and represented in 2-dimensional optimization space including 2 objectives test error and parms.
2. A heuristic is proposed for CNN particles in optimization space; referred as HeuCNN; means that CNNs are assigned their effectiveness in the term of heuristic values.
3. Efficiency of the proposed heuristic is mathematically proven and analyzed in experiments that how it assigns higher heuristics to better CNN particles.
4. Behaviors of CNN particles moving towards directions including vertical (test error objective), horizontal (parms objective) and diagonal (both objectives) are analyzed and the effect of each objective in CNN heuristics is mathematically proven. These analysis are done for both uniform and nonuniform distribution of CNN particles in optimization space.
5. CNN architecture search is modeled as a PSO problem. For this task, new algorithms for particle difference computation, particle velocity computation and particle update rules are proposed.
6. Limitations of the proposed model are discussed and primary insights are directed to address its challenges and proposed as future works.

The rest of this paper is organized as follows: The proposed method is presented in Section 2. Experimental setup and results are reported in Section 3 and 4, respectively. Discussion is presented in sections 5. Limitations of HeuCNN are mentioned and suggested as future works in Section 6. Finally, the paper is concluded in Section 7.

2. Proposed method

In the proposed HeuCNN model, test error and parms are optimized simultaneously. It leads to find the best particles with the lowest test error and parms. Steps of HeuCNN as well as its analysis are presented in following sections.

2.1. Representation of CNN particles

Each CNN architecture is represented as a PSO particle in an array consisting details of its layers and parameters. In our model, 4 types of layers including convolution, max pooling, average pooling and fully-connected layers are investigated. These layers contain the following parameters. Convolution Layer: number of feature maps and kernel size, max pooling and average pooling: kernel size and fully-connected layer: number of neurons. Integration of layers are depicted in Fig. 1 as a particle CNN architecture. Each CNN particle is trained, its test error and parms are represented as a 2-dimensional point in optimization space.

2.2. A heuristic for CNN particles in multi-objective optimization space

CNNs are modeled as particles in 2-dimensional optimization space. A heuristic is proposed to evaluate CNN architectures based in their efficiency. CNN heuristics will be used in the next steps for particle velocity update and particles difference computation. The proposed model for CNN particles in optimization space is shown in Fig. 2. For CNN particle CNN_m three regions including dominated, non-dominated and Don't care are considered. Dominated region (cyan color): CNN_m dominates all CNN particles in this region such as CNN_n , means that they have higher test error and more parms in comparison with CNN_m . Manhattan distance

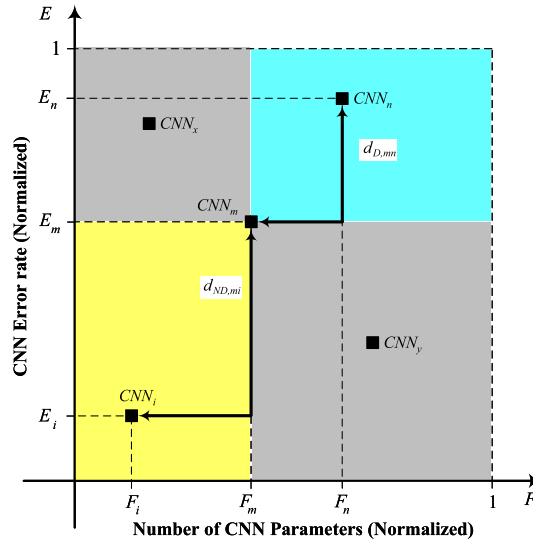


Fig. 2. Representation of CNN Particles in multi-objective optimization space.

between CNN_m and CNN_n ; referred as dominant distance; is represented by $d_{D,mn}$. Non-dominated region (yellow color): CNN_m is dominated by all particles in this region. For CNN particle CNN_i as an example in this region, test error (E_i) and parms (F_i) is lower (better) than E_m and F_m of particle CNN_m . Distance between CNN_m and CNN_i in non-dominated region is represented by $d_{ND,mi}$. Do not care region (gray region): particles in this region such as CNN_x and CNN_y are not dominated by CNN_m and do not dominate CNN_m , mean that there is not any criterion to determine which one is better. The reason is that, in comparison with CNN_m , these CNN particles are better in one objective and worse in another. Particles in Don't care regions are not considered in particle heuristic computation.

Distance between CNN_m and dominated particle CNN_n is defined as $d_{D,mn} = |E_m - E_n| + |F_m - F_n|$, Therefore, distance between CNN_m and all particles in dominated region in cyan color is defined as follows:

$$d_{D,m} = \sum_{n=1}^K f(E_m, E_n, F_m, F_n) \cdot d_{D,mn} = \sum_{n=1}^K f(E_m, E_n, F_m, F_n) \cdot (|E_m - E_n| + |F_m - F_n|) \quad (1)$$

where, $d_{D,m}$ is the dominant distance of particle P_m , K is the number of all particles and $f(\cdot)$ is defined as follows:

$$f(\alpha, \beta, \gamma, \delta) = \begin{cases} 1, & \alpha \leq \beta \text{ and } \gamma \leq \delta \\ 0, & \text{Otherwise} \end{cases} \quad (2)$$

Also, distance between CNN_m and non-dominated particle CNN_i is defined as $d_{ND,mi} = |E_m - E_i| + |F_m - F_i|$, and distance between CNN_m and all particles in non-dominated region in yellow color are computed as:

$$d_{ND,m} = \sum_{i=1}^K g(E_m, E_i, F_m, F_i) \cdot d_{ND,mi} = \sum_{i=1}^K g(E_m, E_i, F_m, F_i) \cdot (|E_m - E_i| + |F_m - F_i|) \quad (3)$$

where, $d_{ND,m}$ is the non-dominant distance of particle CNN_m and $g(\cdot)$ is defined as follows:

$$g(\alpha, \beta, \gamma, \delta) = \begin{cases} 1, & \alpha > \beta \text{ and } \gamma > \delta \\ 0, & \text{Otherwise} \end{cases} \quad (4)$$

CNN's heuristic (CNNH) of CNN_m is defined based on distance of this particle from particles in dominated and non-dominated regions as $CNNH_m = d_{D,m} - d_{ND,m}$ and finally, CNNH is normalized:

$$NCNNH_m = \frac{CNNH_m - \min(CNNH)}{\max(CNNH) - \min(CNNH)} \quad (5)$$

where $CNNH = CNNH_1, CNNH_2, \dots, CNNH_K$.

CNN architectures are considered as particles in multi-objective optimization space. Suppose that K CNN particles exist in optimization space as shown in Fig. 3. This space is decomposed to $M \times N$ regions and one particle in the center of each region is considered as region representator.

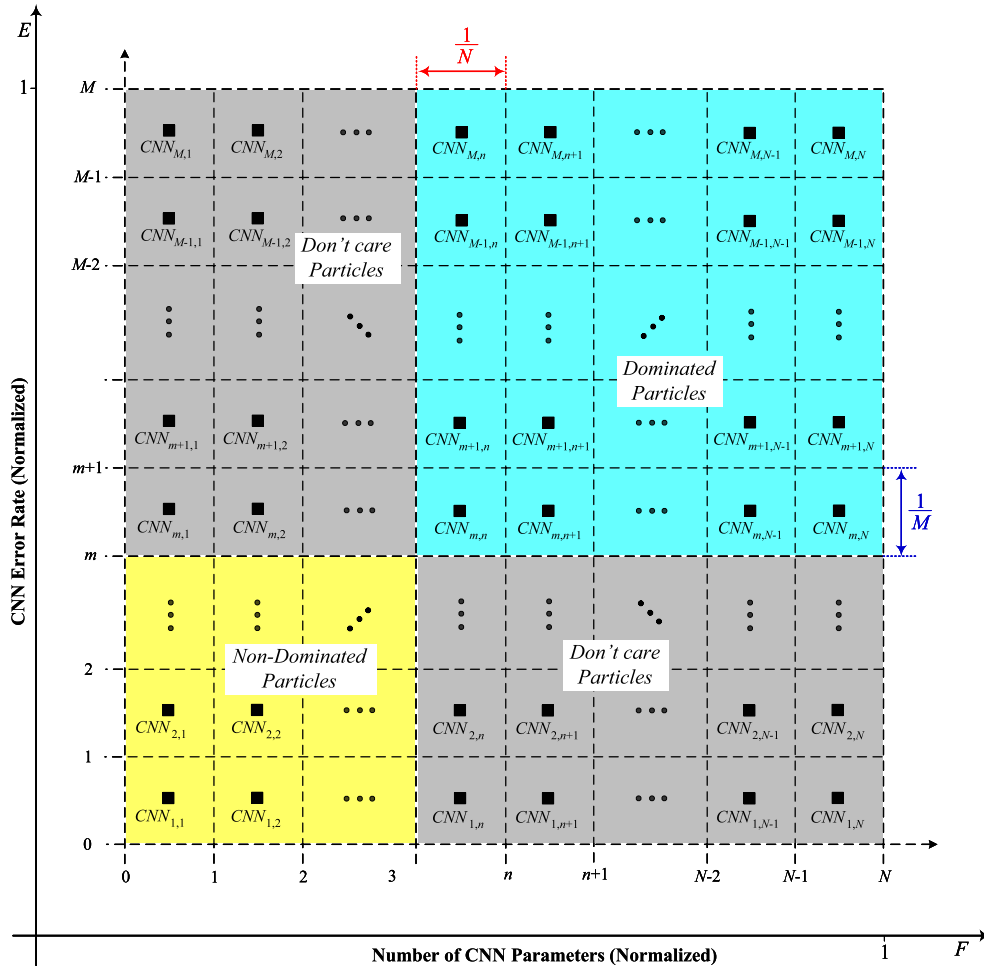


Fig. 3. Proposed model for CNN particles in multi-objective optimization space.

As shown in Fig. 3, for each CNN particle, optimization space is divided into Dominated, non-Dominated and Don't care sub-regions. For particle CNN_{mn} , particles in dominated, non-dominated and Don't care regions lead to better and worse heuristic, respectively. Don't care particles does not have any effect in heuristic computation. All CNN particles in dominated and non-dominated regions affect CNN_{mn} 's heuristic based on their distance from this particle. Therefore,

$$CNNH_{m,n} = a_{m,n} - b_{m,n} \quad (6)$$

where, $a_{m,n}$ and $b_{m,n}$ represent sum distance of CNN_{mn} from particles in Dominated and Non-Dominated regions, respectively. In this analysis, uniform and nonuniform distributions of CNN particles in optimization space are considered.

2.2.1. Uniform distribution of CNN particles

For uniform distribution of CNN particles, optimization space is divided into $M \times N$ regions and each region contains $\frac{K}{MN}$ particles. Distance between CNN_{mn} and all particles in region ij ; which CNN_{ij} is its representator; is computed as follows:

$$d_{(m,n),(i,j)} = \frac{K}{MN} \cdot \left(|m-i| \times \frac{1}{M} + |n-j| \times \frac{1}{N} \right) \quad (7)$$

Therefore, $a_{m,n}$ is computed as:

$$a_{m,n} = \sum_{i=m}^M \sum_{j=n}^N d_{(m,n),(i,j)} = \sum_{i=m}^M \sum_{j=n}^N \frac{K}{MN} \cdot \left(|m-i| \times \frac{1}{M} + |n-j| \times \frac{1}{N} \right) \quad (8)$$

Since $j \geq n$ and $i \geq m$ Eq. (8) can be rewritten as:

$$\begin{aligned}
 a_{m,n} &= \sum_{i=m}^M \sum_{j=n}^N \frac{K}{MN} \cdot \left((i-m) \times \frac{1}{M} + (j-n) \times \frac{1}{N} \right) \\
 &= \frac{K}{M^2 N} \sum_{i=m}^M \sum_{j=n}^N i - \frac{K}{M^2 N} \sum_{i=m}^M \sum_{j=n}^N m + \frac{K}{MN^2} \sum_{i=m}^M \sum_{j=n}^N j - \frac{K}{MN^2} \sum_{i=m}^M \sum_{j=n}^N n \\
 &= \frac{K(N-n+1)}{M^2 N} \cdot \frac{(M-m+1)(M+m)}{2} - \frac{Km(M-m+1)(N-n+1)}{M^2 N} + \frac{K(M-m+1)}{MN^2} \\
 &\quad \cdot \frac{(N-n+1)(N+n)}{2} - \frac{Kn(M-m+1)(N-n+1)}{MN^2}
 \end{aligned} \tag{9}$$

So,

$$a_{m,n} = \frac{K(M-m+1)(N-n+1)(2MN - Mn - Nm)}{2M^2 N^2} \tag{10}$$

Based on distance between two particles in Eq. (7), $b_{m,n}$ is computed as:

$$b_{m,n} = \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} d_{(m,n),(i,j)} = \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} \frac{K}{MN} \cdot \left(|m-i| \times \frac{1}{M} + |n-j| \times \frac{1}{N} \right) \tag{11}$$

Since $m \geq i$ and $n \geq j$ Eq. (11) can be rewritten as:

$$\begin{aligned}
 b_{m,n} &= \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} \frac{K}{MN} \cdot \left((m-i) \times \frac{1}{M} + (n-j) \times \frac{1}{N} \right) \\
 &= \frac{K}{M^2 N} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} m - \frac{K}{M^2 N} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} i + \frac{K}{MN^2} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} n - \frac{K}{MN^2} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} j \\
 &= \frac{Km(m-1)(n-1)}{M^2 N} - \frac{K(n-1)}{M^2 N} \cdot \frac{(m-1)m}{2} + \frac{Kn(m-1)(n-1)}{MN^2} - \frac{K(m-1)}{MN^2} \cdot \frac{(n-1)n}{2}
 \end{aligned} \tag{12}$$

So,

$$b_{m,n} = \frac{K(m-1)(n-1)(Mn + Nm)}{2M^2 N^2} \tag{13}$$

Therefore, final heuristic for particle CNN_{mn} is computed as follows:

$$\begin{aligned}
 CNNH_{m,n} &= a_{m,n} - b_{m,n} \\
 &= \frac{K(M-m+1)(N-n+1)(2MN - Mn - Nm) - K(m-1)(n-1)(Mn + Nm)}{2M^2 N^2}
 \end{aligned} \tag{14}$$

In the proposed model, it is expected that CNN particles which are far away the origin of optimization space, are assigned lower heuristics. To support this conclusion and better visualization of CNN particle heuristics in optimization space, CNN_{mn} is computed for different values of m and n . CNN_{00} means a CNN particle in origin with the lowest test error and fewest parms. In this experiment, CNN heuristics for $M = 10$ and $N = 15$ are computed in 3 different cases (constant n , variable m), (constant m , variable n) and (variable n , variable m). Note that M and N are the dimension of optimization space and m and n represent test error and parms in CNN architecture, respectively. In the first case, n is set to constant values 3, 8 and 13 and m varies from 1 to 10. CNN heuristic are computed by Eq. (14) and the result is shown in Fig. 4-a. In the second case, m is set to constant values 2, 5 and 8 and n varies from 1 to 15. CNN heuristic are computed by Eq. (14) and the result is shown in Fig. 4-b. In the last case, m and n vary from 1 to 10 (see Fig. 4-c).

This analysis demonstrates that in all cases, by increasing m and n , CNN heuristics are decreased linearly. The important behavior of the proposed model is that when one objective (test error or parms) is constant and another is increased (moving to vertical of horizontal directions in optimization space), CNN heuristic is decreased less than the case of simultaneous increment of m and n (moving alongside diameter).

2.2.2. Nonuniform distribution of CNN particles

In this case, particles have nonuniform distribution in optimization space. The probability that particle CNN_{mn} exists in sub-region mn is p_{mn} . Distance between CNN_{mn} and CNN_{ij} is computed as follows:

$$d_{(m,n),(i,j)} = Kp_{i,j} \cdot \left(|m-i| \times \frac{1}{M} + |n-j| \times \frac{1}{N} \right) \tag{15}$$

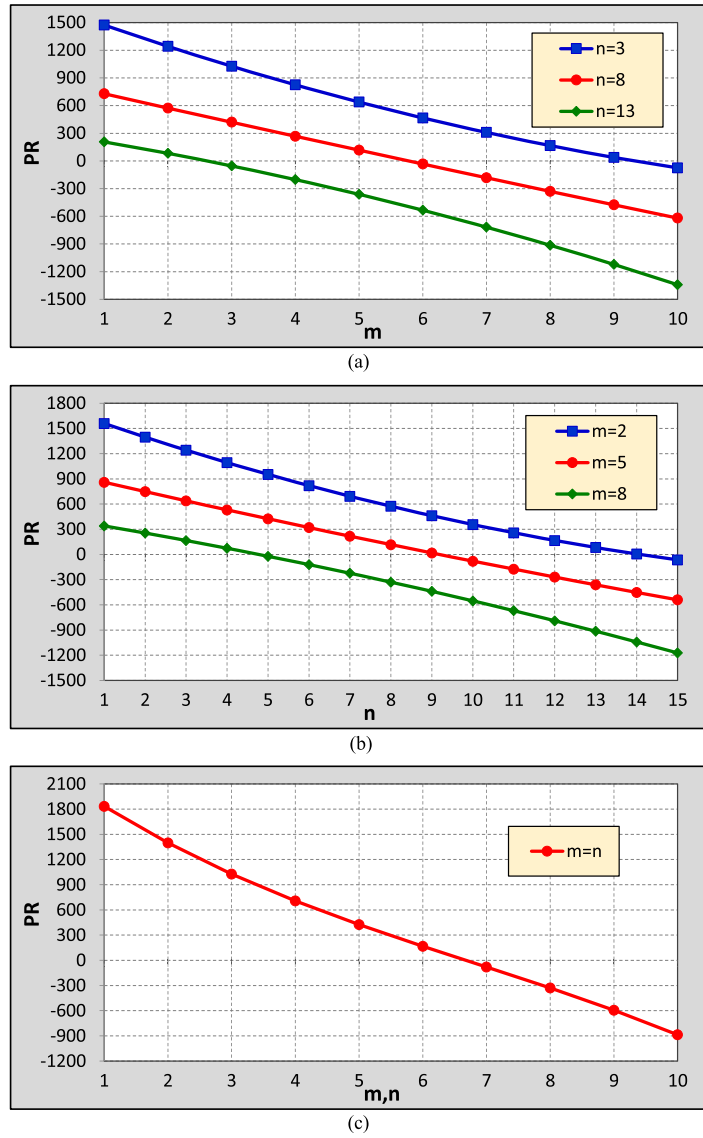


Fig. 4. CNN heuristic analysis in uniform particles distribution.

Sum distance $a_{m,n}$ is defined as:

$$a_{m,n} = \sum_{i=m}^M \sum_{j=n}^N d_{(m,n),(i,j)} = \sum_{i=m}^M \sum_{j=n}^N K p_{i,j} \cdot \left(|m-i| \times \frac{1}{M} + |n-j| \times \frac{1}{N} \right) \quad (16)$$

Since $i \geq m$ and $j \geq n$:

$$\begin{aligned} a_{m,n} &= \sum_{i=m}^M \sum_{j=n}^N K p_{i,j} \cdot \left((i-m) \times \frac{1}{M} + (j-n) \times \frac{1}{N} \right) \\ &= \frac{K}{M} \sum_{i=m}^M \sum_{j=n}^N i p_{i,j} - \frac{Km}{M} \sum_{i=m}^M \sum_{j=n}^N p_{i,j} + \frac{K}{N} \sum_{i=m}^M \sum_{j=n}^N j p_{i,j} - \frac{Kn}{N} \sum_{i=m}^M \sum_{j=n}^N p_{i,j} \end{aligned} \quad (17)$$

By the same way, based on distance computation in Eq. (15), $b_{m,n}$ is computed as follows:

$$b_{m,n} = \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} d_{(m,n),(i,j)} = \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} K p_{i,j} \cdot \left(|m-i| \times \frac{1}{M} + |n-j| \times \frac{1}{N} \right) \quad (18)$$

Since $m \geq i$ and $n \geq j$, Eq. (18) can be rewritten as:

$$\begin{aligned} b_{m,n} &= \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} K p_{i,j} \cdot \left((m-i) \times \frac{1}{M} + (n-j) \times \frac{1}{N} \right) \\ &= \frac{Km}{M} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} p_{i,j} - \frac{K}{M} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} i p_{i,j} + \frac{Kn}{N} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} p_{i,j} - \frac{K}{N} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} j p_{i,j} \end{aligned} \quad (19)$$

The final heuristic for CNN particles is:

$$\begin{aligned} CNNH_{m,n} &= a_{m,n} - b_{m,n} \\ &= \left(\frac{K}{M} \sum_{i=m}^M \sum_{j=n}^N i p_{i,j} - \frac{Km}{M} \sum_{i=m}^M \sum_{j=n}^N p_{i,j} + \frac{K}{N} \sum_{i=m}^M \sum_{j=n}^N j p_{i,j} - \frac{Kn}{N} \sum_{i=m}^M \sum_{j=n}^N p_{i,j} \right) \\ &\quad - \left(\frac{Km}{M} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} p_{i,j} - \frac{K}{M} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} i p_{i,j} + \frac{Kn}{N} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} p_{i,j} - \frac{K}{N} \sum_{i=1}^{m-1} \sum_{j=1}^{n-1} j p_{i,j} \right) \\ &= \frac{K}{M} \sum_{i=1}^M \sum_{j=1}^N i p_{i,j} + \frac{K}{N} \sum_{i=1}^M \sum_{j=1}^N j p_{i,j} - \frac{Km}{M} \sum_{i=1}^M \sum_{j=1}^N p_{i,j} - \frac{Kn}{N} \sum_{i=1}^M \sum_{j=1}^N p_{i,j} \\ &= \frac{K}{M} \sum_{j=1}^N \mu_j + \frac{K}{N} \sum_{i=1}^M \mu_i - \frac{Km}{M} - \frac{Kn}{N} = K \left(\frac{1}{M} \sum_{j=1}^N \mu_j + \frac{1}{N} \sum_{i=1}^M \mu_i - \left(\frac{m}{M} + \frac{n}{N} \right) \right) \end{aligned} \quad (20)$$

where μ_j and μ_i represent the first order momentum of j th column i th row, respectively. Similar with uniform distribution, three cases of m and n are considered and shown in Fig. 5.

Fig. 5 shows that nonuniform distribution of particle in optimization space leads to the similar behavior with uniform distribution. Amounts of heuristic changes in three cases, constant m , constant n and variable m, n are 1323.70, 1380.26 and 2683.88, respectively. It means that moving toward higher test error or higher parms in CNN architectures leads to the same decrement of CNN particle heuristic while moving toward both objectives decreased heuristics significantly (almost twice). The reason is that moving to higher test error and higher parms decreases heuristics with factor $\frac{1}{M}$ and $\frac{1}{N}$ respectively which this factor is $\sqrt{\frac{1}{M^2} + \frac{1}{N^2}}$ when particles are moved in both directions simultaneously.

2.2.3. Efficiency of the proposed heuristic method

To show the effectiveness of the proposed heuristic method, it has been compared with the heuristics based on number of dominated and non-dominated particles (NDNDP). Consider two particles CNN_{mn} and CNN_{ij} in Fig. 6. In this analysis, assigned heuristics by the proposed method and NDNDP are investigated in details. It is clear that CNN_{mn} should receive better heuristics in comparison with particle CNN_{ij} . This parameter can be formulated in relative difference heuristic $\frac{HEU(CNN_{mn}) - HEU(CNN_{ij})}{HEU(CNN_{mn})}$. This analysis is done for the proposed optimization space in Fig. 6 (K particles in $M \times N$ sub-regions).

NDNDP assigns the following heuristic to particle CNN_{mn} :

$$\begin{aligned} HEU_{NDNDP}(CNN_{mn}) &= (N - n + 1)(M - m + 1) \times \frac{K}{MN} - (m - 1)(n - 1) \times \frac{K}{MN} \\ &= \frac{K}{MN} \times (NM - Nm - Mn + M + N) \end{aligned} \quad (21)$$

And for particle CNN_{ij} :

$$HEU_{NDNDP}(CNN_{ij}) = \frac{K}{MN} \times (NM - Ni - Mj + M + N) \quad (22)$$

Relative difference heuristic computed as follows:

$$\frac{HEU_{NDNDP}(CNN_{mn}) - HEU_{NDNDP}(CNN_{ij})}{HEU_{NDNDP}(CNN_{mn})} = 1 - \frac{NM - Ni - Mj + M + N}{NM - Nm - Mn + M + N} \quad (23)$$

Computed relative difference heuristic by the proposed method for these particles are as follows:

$$\begin{aligned} HEU_{HeuCNN}(CNN_{mn}) &= \\ \frac{K(M - m + 1)(N - n + 1)(2MN - Mn - Nm) - K(m - 1)(n - 1)(Mn + Nm)}{2M^2N^2} \end{aligned} \quad (24)$$

$$\begin{aligned} HEU_{HeuCNN}(CNN_{ij}) &= \\ \frac{K(M - i + 1)(N - j + 1)(2MN - Mj - Ni) - K(i - 1)(j - 1)(Mj + Ni)}{2M^2N^2} \end{aligned} \quad (25)$$

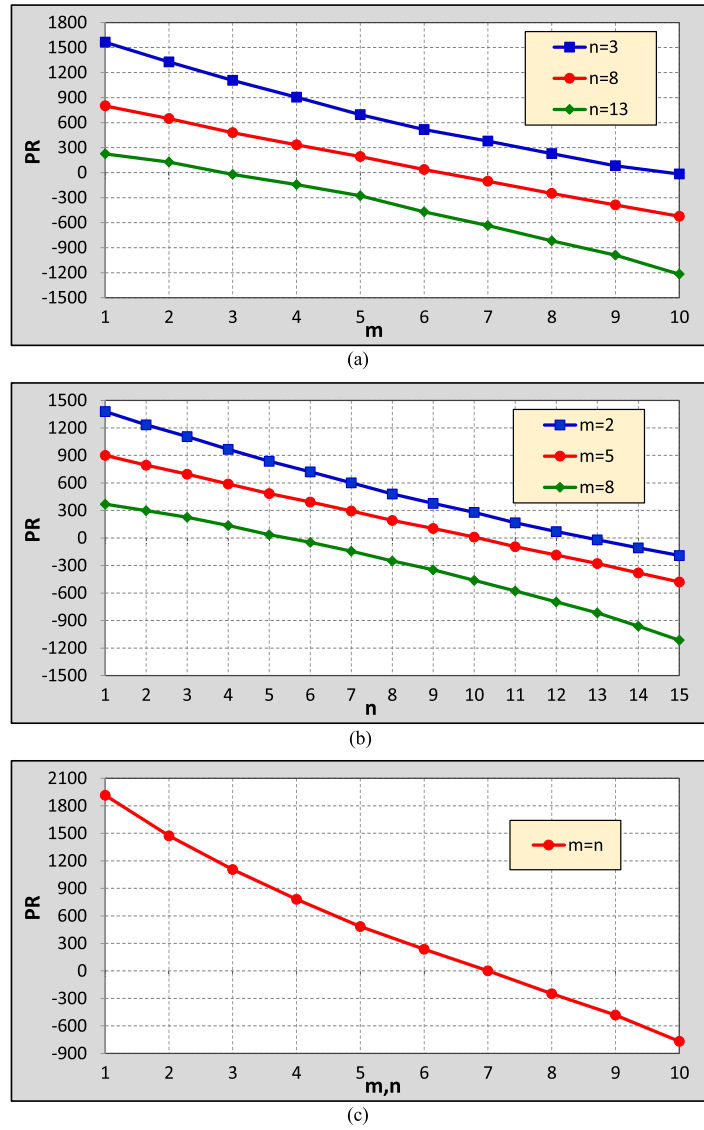


Fig. 5. CNN heuristic analysis in nonuniform particles distribution.

$$\frac{HEU_{HeuCNN}(CNN_{mn}) - HEU_{HeuCNN}(CNN_{ij})}{HEU_{HeuCNN}(CNN_{mn})} = 1 - \frac{(M-i+1)(N-j+1)(2MN - Mj - Ni) - (i-1)(j-1)(Mj + Ni)}{(M-m+1)(N-n+1)(2MN - Mn - Nm) - (m-1)(n-1)(Mn + Nm)} \quad (26)$$

To compare two methods, relative difference heuristic of the proposed HeuCNN is divided to NDNDP as follows:

$$\frac{\frac{HEU_{HeuCNN}(CNN_{mn}) - HEU_{HeuCNN}(CNN_{ij})}{HEU_{HeuCNN}(CNN_{mn})}}{\frac{HEU_{NDNDP}(CNN_{mn}) - HEU_{NDNDP}(CNN_{ij})}{HEU_{NDNDP}(CNN_{mn})}} = 1 - \frac{(M-i+1)(N-j+1)(2MN - Mj - Ni) - (i-1)(j-1)(Mj + Ni)}{(M-m+1)(N-n+1)(2MN - Mn - Nm) - (m-1)(n-1)(Mn + Nm)} \quad (27)$$

$$1 - \frac{NM - Ni - Mj + N}{NM - Nm - Mn + M + N}$$

If Eq. (27) is greater than 1, it means that the proposed method assigns better heuristics to particles CNN_{mn} and CNN_{ij} . To investigate this issue, Eq. (27) is computed for values $K = 2000$, $N = 10$ and $M = 10$. Here, cases (1) $m = n = 1$, $j = 2, 4, 6, 8$, $i = 2, 3, \dots, 9$, (2) $m = n = 1$, $j = 2, 4, 6, 8$, $i = 2, 3, \dots, 9$, (3) $m = 2$, $n = 3$, $j = i + 1 = 3, 4, \dots, 9$, (4) $m = 2$, $n = 2$, $i = j = 3, 4, \dots, 9$, (5)

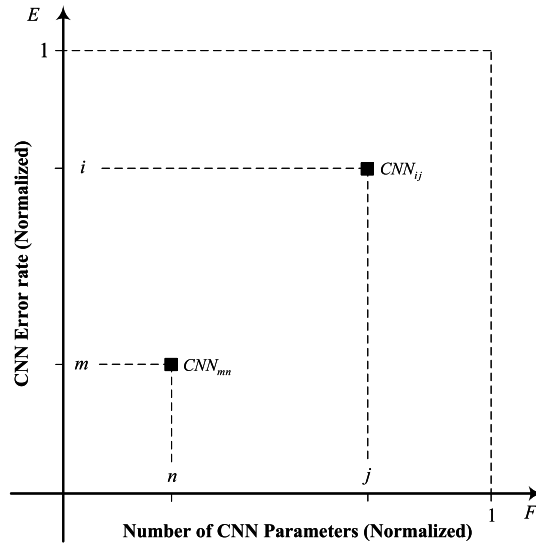


Fig. 6. CNN heuristic efficiency analysis.

$m = 2, n = 3, j = 3, i = 3, 4, \dots, 9$ and (6) $m = 3, n = 2, i = 5, j = 3, 4, \dots, 9$ are considered. Results are shown in Fig. 7 which reveals that in all cases, the proposed method assigns better heuristics (ratio of relative heuristics greater than 1) in comparison with NDNDP.

For nonuniform distribution relative difference heuristics for NDNDP is computed as follows:

$$HEU_{NDNDP}(CNN_{mn}) = K \left(\sum_{k=m}^M \sum_{l=n}^N p_{k,l} - \sum_{k=1}^{m-1} \sum_{l=1}^{n-1} p_{k,l} \right) \quad (28)$$

$$HEU_{NDNDP}(CNN_{ij}) = K \left(\sum_{k=i}^M \sum_{l=j}^N p_{k,l} - \sum_{k=1}^{i-1} \sum_{l=1}^{j-1} p_{k,l} \right) \quad (29)$$

$$\frac{HEU_{NDNDP}(CNN_{mn}) - HEU_{NDNDP}(CNN_{ij})}{HEU_{NDNDP}(CNN_{mn})} = 1 - \frac{\sum_{k=i}^M \sum_{l=j}^N p_{k,l} - \sum_{k=1}^{i-1} \sum_{l=1}^{j-1} p_{k,l}}{\sum_{k=m}^M \sum_{l=n}^N p_{k,l} - \sum_{k=1}^{m-1} \sum_{l=1}^{n-1} p_{k,l}} \quad (30)$$

And for the proposed method:

$$\begin{aligned} & \frac{HEU_{HeuCNN}(CNN_{mn}) - HEU_{HeuCNN}(CNN_{ij})}{HEU_{HeuCNN}(CNN_{mn})} = \\ & 1 - \frac{\frac{1}{M} \sum_{r=1}^M \sum_{s=1}^N r p_{r,s} + \frac{1}{N} \sum_{r=1}^M \sum_{s=1}^N s p_{r,s} - \frac{i}{M} \sum_{r=1}^M \sum_{s=1}^N p_{r,s} - \frac{j}{N} \sum_{r=1}^M \sum_{s=1}^N p_{r,s}}{\frac{1}{M} \sum_{r=1}^M \sum_{s=1}^N r p_{r,s} + \frac{1}{N} \sum_{r=1}^M \sum_{s=1}^N s p_{r,s} - \frac{m}{M} \sum_{r=1}^M \sum_{s=1}^N p_{r,s} - \frac{n}{N} \sum_{r=1}^M \sum_{s=1}^N p_{r,s}} \end{aligned} \quad (31)$$

Finally, ratio of relative difference heuristics for the proposed method and NDNDP is:

$$\begin{aligned} & \frac{HEU_{HeuCNN}(CNN_{mn}) - HEU_{HeuCNN}(CNN_{ij})}{HEU_{HeuCNN}(CNN_{mn})} = \\ & \frac{HEU_{NDNDP}(CNN_{mn}) - HEU_{NDNDP}(CNN_{ij})}{HEU_{NDNDP}(CNN_{mn})} = \\ & 1 - \frac{\frac{1}{M} \sum_{r=1}^M \sum_{s=1}^N r p_{r,s} + \frac{1}{N} \sum_{r=1}^M \sum_{s=1}^N s p_{r,s} - \frac{i}{M} \sum_{r=1}^M \sum_{s=1}^N p_{r,s} - \frac{j}{N} \sum_{r=1}^M \sum_{s=1}^N p_{r,s}}{\frac{1}{M} \sum_{r=1}^M \sum_{s=1}^N r p_{r,s} + \frac{1}{N} \sum_{r=1}^M \sum_{s=1}^N s p_{r,s} - \frac{m}{M} \sum_{r=1}^M \sum_{s=1}^N p_{r,s} - \frac{n}{N} \sum_{r=1}^M \sum_{s=1}^N p_{r,s}} \\ & 1 - \frac{\sum_{k=i}^M \sum_{l=j}^N p_{k,l} - \sum_{k=1}^{i-1} \sum_{l=1}^{j-1} p_{k,l}}{\sum_{k=m}^M \sum_{l=n}^N p_{k,l} - \sum_{k=1}^{m-1} \sum_{l=1}^{n-1} p_{k,l}} \end{aligned} \quad (32)$$

Same experiments are done for nonuniform distribution shown in Fig. 8. This analysis also support that the proposed method assign more effective heuristics to particles in optimization space.

2.3. CNN particles difference computation

In PSO, difference between each particle, pBest and gBest are computed in optimization process. Notations CNN_m , CNN_{pBest} , and CNN_{gBest} are used for current CNN architecture, the best architecture for current CNN in its history, and the best CNN architecture in all generations, respectively. Proposed Algorithm 1 computes the difference between CNN_m and CNN_{pBest} based on CNN_m

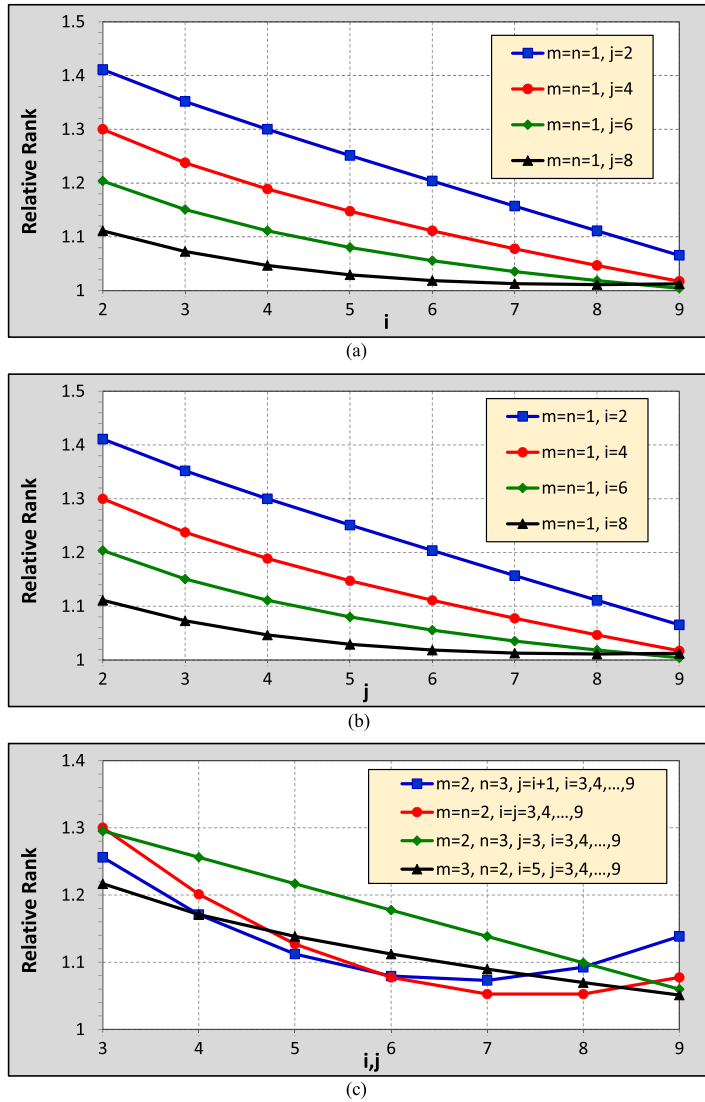


Fig. 7. Ratio of relative heuristics for the proposed HeuCNN and NDNDP in uniform distribution of particles.

heuristic and a random number in interval $[0, 1]$. Note that, $L(\cdot)$ demonstrates layer type of its argument. Based on this algorithm, if CNN_m has a high heuristic, components of $CNN_{m,Diff}$ are selected from CNN_m , otherwise they are selected from other particle (CNN_{pBest} or CNN_{gBest}). It means that worse particles are encouraged to move toward pBest and gBest while better particles move alongside their previous directions.

2.4. CNN particles velocity computation

Velocity of CNN_m is computed based on its difference between CNN_{gBest} and CNN_{pBest} , its heuristic and a random number r between 0 and 1. Steps of the proposed velocity computation of CNN_m is demonstrated in details in Algorithm 2. Please note that if particles are in “Do not Care” regions with respect to each other (in one measure better and in other worse), heuristics of all particles are considered as C_g .

2.5. CNN particles update

CNN particles are updated based on their velocity in Algorithm 3.

The proposed algorithms for difference, velocity and update computation consider particles heuristic as a main decision criterion. Particles with high heuristics are preserved in update process, and particles with low heuristics are enforced to be updated by pBest and gBest.

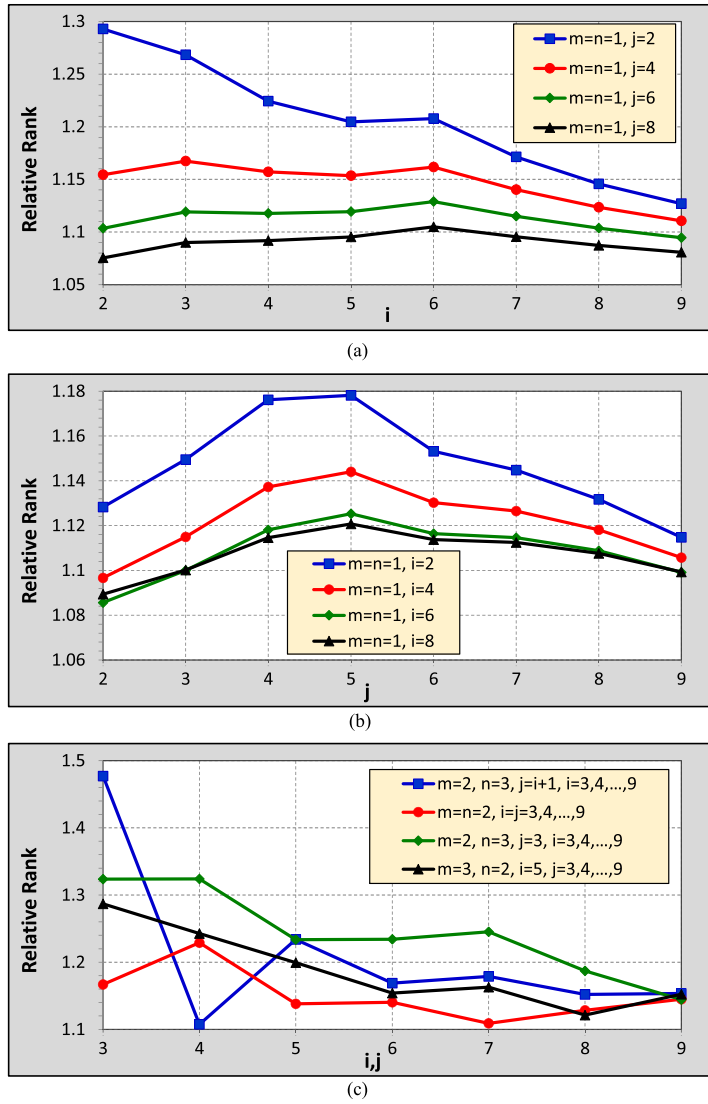


Fig. 8. Ratio of relative heuristics for the proposed HeuCNN and NDNDP in nonuniform distribution of particles.

2.6. CNN optimization algorithm

All proposed methods for heuristic computation, difference, velocity and update rules are integrated in Algorithm 4. First, CNN particles are initialized in steps 1-29. Then, pBest particles, gBest particle and heuristics of all particles in population are computed (steps 30-39). Particles are updated in each iteration of the main loop of optimization in steps 40-57. The best gBest in all iterations is found in steps 58-61. Finally, average test error and average params of the best gBest is computed as final output of the algorithm (steps 62-65).

3. Experimental setup

3.1. Parameter setting

Parameters for swarm optimization, CNN particles initialization and CNN training are configured as follows: $PS = 20$, $max_L = 20$, $min_L = 3$, $min_{KD} = 3 \times 3$, $max_{KD} = 7 \times 7$, $max_{NK} = 256$, $max_{NFC} = 300$, $Conv_P = 0.6$, $Pool_P = 0.3$, $FC_P = 0.1$, $Opt = adam$, $lr = 0.001$, $BS = 32$, $PEE = 1$, $Itr = 10$, $NgBestTr = 10$, $gBestEE = 100$, $C_g = 0.5$, $PKD = 3 \times 3$, $CS = 1 \times 1$, $PS = 2 \times 2$ and $DropoutRate = 0.5$.

Algorithm 1 Proposed CNN Particles Difference Computation.**Input:** CNN_m and CNN_{pBest} .**Output:** $CNN_{m,Diff}$: Difference between CNN_m and CNN_{pBest} .

```

1: For all corresponding layers in  $CNN_m$  and  $CNN_{pBest}$  do the following comparisons:
2:   if  $L(CNN_m) == L(CNN_{pBest})$  then
3:      $L(CNN_{m,Diff}) = L(CNN_m)$ ;
4:   else if  $L(CNN_m) \sim L(CNN_{pBest})$  then
5:      $r = rand()$ ;
6:     if  $r \leq CNNH_m$  then
7:        $L(CNN_{m,Diff}) = L(CNN_m)$ ;
8:     else
9:        $L(CNN_{m,Diff}) = L(CNN_{pBest})$ ;
10:  else if  $L(CNN_m) == \phi$  and  $L(CNN_{pBest}) \sim \phi$  then
11:     $L(CNN_{m,Diff}) = L(CNN_{pBest})$ ;
12:  else
13:     $L(CNN_{m,Diff}) = \phi$ ;

```

Algorithm 2 Proposed CNN Particles Velocity Computation.**Input:** CNN_m , CNN_{gBest} and CNN_{pBest} .**Output:** $CNN_{m,V}$: Velocity of CNN_m .

```

1: For all corresponding layers in  $CNN_m$ ,  $[CNN_{pBest} - CNN_m]$  and  $[CNN_{gBest} - CNN_m]$  do the following tasks:
2:   if  $L(CNN_m).Type == L(CNN_{pBest}).Type == L(CNN_{gBest}).Type$  then
3:      $r1 = rand()$ ;
4:     if  $r1 \leq C_g$  then
5:        $L(CNN_{m,V}) = L(CNN_{gBest})$ ;
6:     else
7:        $L(CNN_{m,V}) = L(CNN_{pBest})$ ;
8:   else
9:      $r = rand()$ ;
10:    if  $r \leq CNNH_m$  then
11:       $[CNN_{pBest} - CNN_m]$  is considered for velocity computation:
12:      if  $L[CNN_{pBest} - CNN_m] \sim \phi$  then
13:         $L(CNN_{m,V}) = L([CNN_{pBest} - CNN_m])$ ;
14:      else
15:         $L(CNN_{m,V}) = \phi$ ;
16:    else
17:       $[CNN_{gBest} - CNN_m]$  is considered for velocity computation:
18:      if  $L[CNN_{gBest} - CNN_m] \sim \phi$  then
19:         $L(CNN_{m,V}) = L([CNN_{gBest} - CNN_m])$ ;
20:      else
21:         $L(CNN_{m,V}) = \phi$ ;

```

Algorithm 3 Proposed CNN Particles Update.**Input:** CNN_m and $CNN_{m,V}$.**Output:** $CNN_{m,Udated}$: Updated of CNN_m .

```

1: if  $L(CNN_{m,V}) == \phi$  then
2:    $L(CNN_{m,Udated}) = \phi$ ;
3: else
4:    $L(CNN_{m,Udated}) = L(CNN_{m,V})$ ;
5:  $k = 0$ ;
6: for  $i = 1$  to Number of layers in  $CNN_{m,Udated}$  do
7:   if  $i$ th layer of  $CNN_{m,Udated} == \text{pooling}$  then
8:      $k = k + 1$ ;
9:     if  $k > 3$  then
10:       $i$ th layer of  $CNN_{m,Udated} = \phi$ ;

```

3.2. Datasets

Since many CNN models have been evaluated on publicly available datasets, HeuCNN is applied to 9 public datasets including Rectangles (Rect), Rectangles-Images (Rect-I), Convex, MNIST, MNIST with Background images (MNIST-BI), MNIST with random background (MNIST-RB), MNIST rotated digits (MNIST-RD), MNIST rotated with background images (MNIST-RDBI) and MNIST-Fashion. Description of these datasets have been reported in Table 1.

3.3. Peer competitive methods

Proposed CNN models have reported different measures of evaluation. Since HeuCNN is an optimization method with two objectives including classification test error and number of parameters, CNN models which have reported these measures (in selected

Algorithm 4 Proposed HeuCNN.

Notations: $NL(CNN_i)$: number of layers in i th CNN particle, $rand(a, b)$: return a random number between a and b , $Layers[1].Type$: type of i th layer, $Layers[1].KerNum$: number of kernels in i th layer, $Layers[1].KerDim$: dimension of kernels in i th layer, $size(a)$: number of element in vector a , $CNNTrain$: train a CNN model with training data, $Evaluate$: evaluate CNN models with test data, $ComputeHeuristic$: compute heuristic for CNN particles based on the proposed method,
Input: population size (PS), maximum number of layers (max_L), minimum number of layers in CNN architectures (min_L), minimum kernel dimension (min_{KD}), maximum kernel dimension (max_{KD}), maximum number of kernels (max_{NK}), Training data (TrD) and Test data (TD) vectors and labels ($Labels$), number of labels (n_{out}), maximum neurons in fully-connected layer (max_{NFC}); probability of adding convolution, pooling and fully-connected layers: ($Conv_p$), ($Pool_p$) and (FC_p);
Opt: optimization method to train CNNs, lr : learning rate, BS : batch size, $P EE$: particle evaluation epochs, $Iter$: number of iterations for PSO optimization process, $NgBestTr$: number of gBest training, $gBestEE$: gBest evaluation epochs, pooling kernel dimension (PKD), stride in convolution layer (CS), stride in pooling layer (PS).

Output: gBest in all iterations, average test error and parms of gBest.

```

1: for  $i = 1$  to  $PS$  do
2:    $NL(CNN_i) = round(L \times rand(0, 1))$ 
3:   if  $NL(CNN_i) < min_L$  then
4:      $NL(CNN_i) \leftarrow min_L$ ;
5:    $Layers[1].Type \leftarrow Convolution$ 
6:    $Layers[1].KerNum \leftarrow rand(1, max_{NK})$ 
7:    $Layers[1].KerDim \leftarrow rand(min_{KD}, max_{KD})$ 
8:   for  $j = 2$  to  $NL(CNN_i)$  do
9:     if  $Layers[j-1].Type = "FullyConnected"$  then
10:       $Layers[j].Type \leftarrow FullyConnected$ ;
11:       $Layers[j].NumNeu \leftarrow max_{NFC}$ ;
12:     else
13:        $r \leftarrow rand(0, 1)$ ;
14:       if  $r \leq Conv_p$  then
15:          $Layers[j].Type \leftarrow Convolution$ 
16:          $Layers[j].KerNum \leftarrow rand(1, max_{NK})$ 
17:          $Layers[j].KerDim \leftarrow rand(min_{KD}, max_{KD})$ 
18:          $Layers[j].Stride \leftarrow CS$ 
19:       else if  $r > Conv_p$  &  $r \leq Conv_p + Pool_p$  then
20:          $Layers[j].Type \leftarrow Pooling(Select\ Average\ or\ Max\ with\ the\ probabilities\ of\ 50\%)$ ;
21:          $Layers[j].KerDim \leftarrow PKD$ ;
22:          $Layers[j].Stride \leftarrow PS$ 
23:       else
24:          $Layers[j].Type \leftarrow FullyConnected$ ;
25:          $Layers[j].NumNeu \leftarrow max_{NFC}$ ;
26:    $Layers[NL(CNN_i)].Type \leftarrow FullyConnected$ ;
27:    $Layers[NL(CNN_i)].NumNeu \leftarrow (size(labels))$ ;
28:    $CNN_i.Layer \leftarrow Layers$ ;
29: Save  $S = \{CNN_1, ..., CNN_N\}$ 
30: for  $i = 1$  to  $PS$  do
31:    $CNN_i.pBest \leftarrow CNN_i$ ;
32:    $Model \leftarrow CNNTrain(CNN_i, TrD, Opt, lr, BS, PEE)$ ;
33:    $[CNN_i.TestError, CNN_i.Parameters] \leftarrow Evaluate(Model, TD)$ ;
34: Normalize test error and parms of all CNN particles.
35: for  $i = 1$  to  $PS$  do
36:    $Heuristic(CNN_i) \leftarrow ComputeHeuristic(CNN_i)$ ;
37: Normalize heuristic of CNN particles.
38:  $gBest \leftarrow CNN_i$ ; where  $(CNN_i.TestError + CNN_i.Parameters)$  is minimum.
39:  $gBests[1] \leftarrow gBest$ ;
40: for  $itr = 1$  to  $Iter$  do
41:   for  $i = 1$  to  $PS$  do
42:     Compute difference between  $CNN_i$  and  $CNN_i.pBest$  based on Algorithm 1.
43:     Compute difference between  $CNN_i$  and  $gBest[itr]$  based on Algorithm 1.
44:     Compute velocity of  $CNN_i$  based on Algorithm 2.
45:     Update  $CNN_i$  based on Algorithm 3.
46:      $Model \leftarrow CNNTrain(CNN_i, TrD, Opt, lr, BS, PEE)$ ;
47:      $[CNN_i.TestError, CNN_i.Parameters] \leftarrow Evaluate(Model, TD)$ ;
48:   for  $i = 1$  to  $PS$  do
49:      $Heuristic(CNN_i) \leftarrow ComputeHeuristic(CNN_i)$ ;
50:   Normalize heuristic of CNN particles.
51:   for  $i = 1$  to  $PS$  do
52:     if  $CNN_i.TestError + CNN_i.Parameters < CNN_i.pBest.TestError + CNN_i.pBest.Parameters$  then
53:        $CNN_i.pBest \leftarrow CNN_i$ ;
54:        $CNN_i.pBest.TestError \leftarrow CNN_i.TestError$ ;
55:        $CNN_i.pBest.Parameters \leftarrow CNN_i.Parameters$ ;
56:    $gBest \leftarrow CNN_i$ ; where  $(CNN_i.TestError + CNN_i.Parameters)$  is minimum.
57:    $gBests[itr] \leftarrow gBest$ ;
58: for  $i = 1$  to  $size[gBests]$  do
59:    $Model \leftarrow CNNTrain(gBests[i], TrD, Opt, lr, BS, PEE)$ ;
60:    $[gBests[i].TestError, gBests[i].Parameters] \leftarrow Evaluate(Model, TD)$ ;
61:  $gBest \leftarrow CNN_i$ ; where  $(CNN_i.TestError + CNN_i.Parameters)$  is minimum.
62: for  $i = 1$  to  $NgBestTr$  do
63:    $Model \leftarrow CNNTrain(gBest, TrD, Opt, lr, BS, gBestEE)$ ;
64:    $[TestError[i], Parameters[i]] \leftarrow Evaluate(Model, TD)$ ;
65: Return  $gBest$ ,  $Average(TestError)$ ,  $Average(Parameters)$ ;

```

Table 1
Description of datasets.

#	Dataset	Image Size	# Training	# Test	# Labels
1	Rect	32×28	1,200	50,000	2
2	Rect-I	34×28	12,000	50,000	2
3	Convex	35×28	8,000	50,000	2
4	MNIST	28×28	60,000	10,000	10
5	MNIST-RD	29×28	12,000	50,000	10
6	MNIST-RB	30×28	12,000	50,000	10
7	MNIST-BI	31×28	12,000	50,000	10
8	MNIST-RDBI	32×28	12,000	50,000	10
9	MNIST-Fashion	36×28	60,000	10,000	10

Table 2

Primary evaluation of HeuCNN for 5 runs in Convex, Rect, Rect-I, MNIST-RD, MNIST-RDBI, MNIST-RB, MNIST-BI, MNIST and MNIST-Fashion datasets.

Convex				Rect				Rect-I			
Acc.	Error	Parms	Epochs	Acc.	Error	Parms	Epochs	Acc.	Error	Parms	Epochs
98.492	1.508	1027055	10	100	0	594740	10	98.476	1.524	563575	5
96.962	3.038	981013	10	99.688	0.312	1371	10	98.686	1.314	824604	5
95.480	4.520	559234	5	99.708	0.292	3100	10	98.332	1.668	1396440	5
97.116	2.884	726838	5	99.950	0.050	437668	10	98.892	1.108	244567	5
97.602	2.398	72264	5	100	0	73377	10	97.840	2.160	797858	5

MNIST-RD				MNIST-RDBI				MNIST-RB			
Acc.	Error	Parms	Epochs	Acc.	Error	Parms	Epochs	Acc.	Error	Parms	Epochs
96.462	3.538	693146	5	87.734	12.266	717170	5	97.504	2.496	204058	5
97.020	2.980	343832	5	85.666	14.334	64581	5	98.452	1.548	474598	5
96.670	3.330	856332	5	90.078	9.922	786453	5	98.010	1.990	1307103	5
96.440	3.560	617331	5	85.132	14.868	273301	5	98.254	1.746	439341	5
96.232	3.768	471742	5	88.528	11.472	1794829	5	98.356	1.644	988024	5

MNIST-BI				MNIST				MNIST-Fashion			
Acc.	Error	Parms	Epochs	Acc.	Error	Parms	Epochs	Acc.	Error	Parms	Epochs
97.418	2.582	764077	5	99.560	0.440	108372	1	93.810	6.190	741355	1
97.834	2.166	154430	5	99.570	0.430	315686	1	94.040	5.960	659354	1
97.498	2.502	151054	5	99.590	0.410	496561	1	94.660	5.340	556639	1
97.558	2.442	1631524	5	99.620	0.380	287639	1	93.430	6.570	819270	1
97.388	2.612	1577603	5	99.630	0.370	133766	3	93.850	6.150	531118	3

datasets) are considered for comparison. Therefore, LeNet-1, LeNet-4, LeNet-5, DropConnect, CAE-1, CAE-2, PCANet-2, RandNet-2, LDANet-2, TIRBM, PGBM + DN-1, ScatNet-2, SVM + RBF, SVM + Poly, NNet, SAA-3, DBN-3, evoCNN(best), evoCNN(mean), IPPSO(best), IPPSO(mean), psoCNN(best), psoCNN(mean), BQ-CNN(best), BQ-CNN(mean), MLP 256-128-100, AlexNet, 3CONV + 3FC, VGG16, GoogLeNet, MobileNet, RCNN-96, RCNN-128, RCNN-160, Human Performance, GRU + SVM, HOG + SVM, and CPOCNN; discussed in Introduction Section; are used as peer competitive methods.

4. Experimental results

This section reports various experiments applied to HeuCNN and other state-of-the-art methods.

4.1. Primary evaluation

In the first experiment, the proposed HeuCNN has been executed in all datasets for 5 different runs. Some runs have the same number of epochs and some have different. The purpose of these experiments is to evaluate HeuCNN in the term of accuracy, test error and parms in Rect, Rect-I, MNIST-RD, MNIST-RDBI, MNIST-RB, MNIST-BI, MNIST and MNIST-Fashion datasets as reported in Table 2. In Convex dataset, 10 epochs has the test error of 1.508 while the best parms 72264 is appeared in 5 epochs execution. In other datasets, HeuCNN finds CNNs with different test errors and parms although it is executed with the same number of epochs. This property stems from this fact that CNNs are optimized by random initializations which leads to different results. In MNIST-Fashion dataset, 1-epoch CNN with test error and parms of 5.34 and 556639, respectively, is the best architecture means that higher number of epochs does not guarantee the better performance.

Table 3

Test errors and parms of psoCNN and HeuCNN in Convex, Rect, Rect-I, MNIST, MNIST-RD, MNIST-RDBI, MNIST-RB and MNIST-Fashion datasets.

Dataset	HeuCNN				psoCNN				Mean	Best
	Mean		Best		Mean		Best			
	Error	Parms	Error	Parms	Error	Parms	Error	Parms	Error	Parms
Convex	2.87	673281	1.51	1027055	3.90	NA	1.70	1677161	1.03%(+)	0.19%(+)
Rect	0.13	222051	0.00	73377	0.34	NA	0.03	1069296	0.20%(+)	0.07%(+)
Rect-I	1.55	765409	1.11	244567	3.94	NA	2.22	1345500	2.38%(+)	1.11%(+)
MNIST-RD	3.44	596477	2.98	343832	6.42	NA	3.58	1544376	2.98%(+)	0.60%(+)
MNIST-RDBI	12.57	727267	9.92	786453	20.98	NA	14.28	2020534	8.40%(+)	4.35%(+)
MNIST -RB	1.88	682625	1.55	474598	2.53	NA	1.79	3992430	0.64%(+)	0.24%(+)
MNIST -BI	2.46	855738	2.17	154430	2.40	NA	1.90	4107995	0.06%(-)	0.26%(-)
MNIST	0.41	268405	0.37	133766	0.44	NA	0.32	2454643	0.03%(+)	0.05%(-)
MNIST-Fashin	6.04	661547	5.34	659354	5.90	NA	5.53	2320000	0.14%(-)	0.19%(+)

4.2. Comparison with the most similar method

Among all methods discussed in literature review, psoCNN is the most similar method with HeuCNN since they are both optimized by PSO with almost same encoding scheme. In both methods, “best” and “mean” cases are considered for evaluation. Suppose that optimization method is executed in K runs. Each run R_i ($i = 1, \dots, K$) has its own best particle $gBest_i$. “best” case is the best $gBest$ among all runs while “mean” case is average of all $gBest_i$ particles. For psoCNN, architectures of $gBest$ particles in all runs have not been reported. Therefore, number of parameters for “mean” case is not available (NA). As indicated in Table 3, with regard to test error measure in “best” case, HeuCNN outperforms psoCNN with improvement of 1.03%(+), 0.20%(+), 2.38%(+), 2.98%(+), 8.40%(+), 0.64%(+), 0.03%(+) in Convex, Rect, Rect-I, MNIST-RD, MNIST-RDBI, MNIST-RB and MNIST, respectively. In MNIST-BI and MNIST-Fashion psoCNN has better performances with slight improvement of 0.06%(-) and 0.14%(-), respectively. For the best-case comparison, HeuCNN is superior method in 7 datasets out of 9. Interested achievement of HeuCNN is the number of parameters. In all cases, HeuCNN finds optimized CNNs with lower parms. Parms of CNNs found by HeuCNN and psoCNN in MNIST-BI are 154430 and 4107995, respectively. It means that HeuCNN finds an optimized CNN architecture with 0.03% computation cost of the CNN found by psoCNN.

4.3. Comparison in the term of test error

As CNN models for image classification have been reviewed in Introduction section, there are two types of CNNs including CNNs with constant architectures and CNNs with optimized architectures. Test errors of HeuCNN and other methods are reported in Table 4. Bold test errors represent the lowest ones. HeuCNN achieves the lowest test errors 1.54, 1.88, 2.46, 9.92, 12.57, 0, 0.13, 1.10, 1.55 and 1.50 in MNIST-RB(best), MNIST-RB(mean), MNIST-BI(mean), MNIST-RDBI(best), MNIST-RDBI(mean), Rect(best), Rect(mean), Rect-I(best), Rect-I(mean) and Convex(best) datasets, respectively. DropConnect, BQ-CNN(best), BQ-CNN(mean), psoCNN(best) and BQ-CNN(mean) have the best test errors 0.21, 2.0, 3.0, 1.90, 2.31 in MNIST, MNIST-RD, MNIST-RD, MNIST-BI and Convex datasets, respectively. HeuCNN is the best method among 26 CNN models in 6 datasets out of 8. In MNIST and MNIST-RD datasets, HeuCNN is the third and second method, respectively. Please note that comparison of HeuCNN is done for best and mean cases with methods which have reported these cases. In MNIST-BI dataset for example, psoCNN(best) and psoCNN(mean) achieved 1.90 and 2.52 while HeuCNN(best) and HeuCNN(mean) has test errors 2.16 and 2.46, respectively. In these cases, psoCNN(best) and HeuCNN(mean) are winners in pairwise comparison.

4.4. Comparison in the term of parms

Number of parameters; referred as parms; is the second objective in HeuCNN. Table 5 reports parms in Rect, Rect-I, MNIST-RD, MNIST-RDBI, MNIST-RB, MNIST-BI, MNIST, Convex and MNIST-Fashion datasets. Since the majority of CNN models (constant or optimized) have not reported their architectures, comparison for this measure has been limited to 12 models. The last two rows in bold format are parms of HeuCNN. For the best case, number of parameters in the “best” particle in all runs is calculated. For “mean” case, average number of parameters in best particles in each run is computed and reported. Other reported numbers in this table represent parms of HeuCNN over each method. For example, 0.12 in MNIST-BI dataset means $\frac{\text{parms(HeuCNN)}}{\text{parms(IPPSO)}} = 0.12$. In contrast with test error measure (in which best is better than mean), in parms evaluation, best case may have higher parms than mean case. It is due to this fact that the best particle is not a particle with the minimum of parms. As it can be concluded from this table, HeuCNN reaches to CNNs with very low parms and in some cases it has only 1 percent parms of conventional CNNs (GoogleNet in Rect dataset). Among 86 cases for parms comparison, HeuCNN finds CNNs with lower parms in 81 cases. In 5 bold cases, 3CONV + 3FC and RCNN-96 achieve better parms (fraction is greater than 1).

Table 4

Test errors of HeuCNN and other methods in Rect, Rect-I, MNIST-RD, MNIST-RDBI, MNIST-RB, MNIST-BI, MNIST and Convex datasets.

Model	MNIST	MNIST-RD	MNIST-RB	MNIST-BI	MNIST-RDBI	Rect	Rect-I	Convex
LeNet-1	1.70%	–	–	–	–	–	–	–
LeNet-4	1.10%	–	–	–	–	–	–	–
LeNet-5	0.95%	–	–	–	–	–	–	–
RCNN	0.31%	–	–	–	–	–	–	–
DropConnect	0.21%	–	–	–	–	–	–	–
CAE-1	2.83%	11.59%	13.57%	16.70%	48.10%	1.48%	21.86%	–
CAE-2	2.48%	9.66%	10.90%	15.50%	45.23%	1.21%	21.54%	–
PCANet-2	1.06%	8.52%	6.85%	11.55%	35.86%	0.49%	13.39%	4.19%
RandNet-2	1.27%	8.47%	13.47%	11.65%	43.69%	0.09%	17.00%	5.45%
LDANet-2	1.40%	4.52%	6.81%	12.42%	38.54%	0.14%	16.20%	7.22%
TIRBM	–	4.20%	–	–	35.50%	–	–	–
PGBM + DN-1	–	–	6.08%	12.25%	36.76%	–	–	–
ScatNet-2	1.27%	7.48%	12.30%	18.40%	50.48%	–	–	6.50%
SVM + RBF	3.03%	10.38%	14.58%	22.61%	32.62%	–	–	19.13%
SVM + Poly	3.69%	13.61%	16.62%	24.01%	37.59%	–	–	19.82%
NNet	4.69%	17.62%	20.04%	27.41%	42.17%	–	–	32.25%
SAA-3	3.46%	11.43%	11.28%	23.00%	24.09%	–	–	18.41%
DBN-3	3.11%	12.30%	6.73%	16.31%	28.51%	–	–	18.63%
evoCNN (best)	1.18%	5.22%	2.80%	4.53%	35.03%	0.01%	5.03%	4.82%
evoCNN (mean)	1.28%	5.46%	3.59%	4.62%	37.38%	0.01%	5.97%	5.39%
IPPSO (best)	1.13%	–	–	–	33.00%	–	–	8.48%
IPPSO (mean)	1.21%	–	–	–	34.50%	–	–	12.06%
psoCNN (best)	0.32%	3.59%	1.79%	1.90%	14.28%	0.03%	2.22%	1.70%
psoCNN (mean)	0.44%	6.42%	2.53%	2.52%	20.98%	0.34%	3.94%	3.90%
BQ-CNN (best)	1.00%	2.00%	3.00%	5.00%	14.20%	–	–	1.65%
BQ-CNN (mean)	1.00%	3.00%	3.00%	5.00%	16.94%	–	–	2.31%
CPOCNN	0.27%	4.56%	2.08%	3.83%	13.09%	–	3.84%	2.20%
HeuCNN (best)	0.37%	2.98%	1.54%	2.16%	9.92%	0%	1.10%	1.50%
HeuCNN (mean)	0.40%	3.43%	1.88%	2.46%	12.57%	0.13%	1.55%	2.86%

Table 5

Params of HeuCNN and other methods in Rect, Rect-I, MNIST-RD, MNIST-RDBI, MNIST-RB, MNIST-BI, MNIST and Convex and MNIST-Fashion datasets.

Model	MNIST	MNIST-RD	MNIST-RB	MNIST-BI	MNIST-RDBI	Rect	Rect-I	Convex	MNIST-Fashion
IPPSO	–	–	–	0.12	0.65	–	–	–	–
evoCNN	–	–	–	–	–	–	–	–	0.101
psoCNN	0.054	0.223	0.119	0.038	0.389	0.069	0.182	0.612	0.284
MLP 256-128-100	–	–	–	–	–	–	–	–	0.22
AlexNet	0.002	0.006	0.008	0.002	0.013	0.001	0.004	0.016	0.011
3CONV + 3FC	0.268	0.688	0.949	0.309	1.573	0.147	0.489	2.054	1.319
VGG16	0.005	0.013	0.018	0.006	0.03	0.003	0.009	0.04	0.025
GoogLeNet	0.006	0.015	0.021	0.007	0.034	0.003	0.011	0.045	0.029
MobileNet	0.033	0.086	0.119	0.039	0.197	0.018	0.061	0.257	0.165
RCNN-96	0.2	0.513	0.708	0.23	1.174	0.11	0.365	1.533	0.984
RCNN-128	0.112	0.289	0.399	0.13	0.661	0.062	0.206	0.863	0.554
RCNN-160	0.072	0.185	0.255	0.083	0.423	0.039	0.131	0.552	0.354
HeuCNN (best)	133766	343832	474598	154430	786453	73377	244567	1027055	659354
HeuCNN (mean)	268405	596477	682625	855738	727267	222051	765409	673281	661547

4.5. Architecture of the best model

Optimized architectures in each dataset found by HeuCNN are illustrated in Table 6. These architectures are the best architecture with the lowest error rate and fewest params in optimization space. In each architecture, layer type, parameters, kernel size (KS) and output dimension of each layer are reported. Interested conclusion from this table is that, in the literature, expert's knowledge of domain recommends CNNs with one fully-connected layer. Although in optimization process CNNs are allowed to have any number of FC layers, one fully-connected layer is selected by HeuCNN for optimized CNNs. It means that HeuCNN discovers high-performance architectures without any background knowledge of domain.

4.6. Evaluation of MNIST-fashion

Many CNN models use options such as data augmentation, parallel layers, skip connection, dropout and batch normalization to improve their performance. These cases lead to cost overheads. It is worth mentioning that HeuCNN does not use these options and has the simplest architecture. To show the effect of such options in HeuCNN, it is evaluated with and without dropout and batch

Table 6

Optimized architecture for each dataset found by HeuCNN.

Convex		Rectangles-images		MNIST-RD		MNIST-RDBI	
Layers	Parameters	Layers	Parameters	Layers	Parameters	Layers	Parameters
Convolution	KS: 6×6 Output: 134	Convolution	KS: 5×5 Output: 54	Convolution	KS: 5×5 Output: 58	Convolution	KS: 7×7 Output: 90
Max Pooling	KS: 3×3 Strides: 2×2	Convolution	KS: 7×7 Output: 55	Convolution	KS: 4×4 Output: 81	Convolution	KS: 6×6 Output: 82
Convolution	KS: 7×7 Output: 72	Convolution	KS: 4×4 Output: 19	Convolution	KS: 5×5 Output: 37	Convolution	KS: 3×3 Output: 62
Convolution	KS: 4×4 Output: 71	Average Pooling	KS: 3×3 Strides: 2×2	Convolution	KS: 4×4 Output: 40	Max Pooling	KS: 3×3 Strides: 2×2
Convolution	KS: 6×6 Output: 111	Convolution	KS: 3×3 Output: 31	Average Pooling	KS: 3×3 Strides: 2×2	Convolution	KS: 5×5 Output: 79
Convolution	KS: 3×3 Output: 41	Convolution	KS: 6×6 Output: 32	Convolution	KS: 4×4 Output: 62	Convolution	KS: 7×7 Output: 82
Convolution	KS: 5×5 Output: 40	Average Pooling	KS: 3×3 Strides: 2×2	Convolution	KS: 4×4 Output: 43	Average Pooling	KS: 3×3 Strides: 2×2
Max Pooling	KS: 3×3 Strides: 2×2	Convolution	KS: 5×5 Output: 45	Convolution	KS: 3×3 Output: 41	Fully-Connected	Output neurons: 64
Convolution	KS: 3×3 Output: 73	Fully-Connected	Output neurons: 2	Fully-Connected	Output neurons: 10	Fully-Connected	Output neurons: 10
Convolution	KS: 5×5 Output: 39						
Fully-Connected	Output neurons: 2						

MNIST-BI		Rect		MNIST-RB		MNIST	
Layers	Parameters	Layers	Parameters	Layers	Parameters	Layers	Parameters
Convolution	KS: 5×5 Output: 46	Convolution	KS: 4×4 Output: 63	Convolution	KS: 4×4 Output: 181	Convolution	KS: 6×6 Output: 165
Convolution	KS: 7×7 Output: 36	Convolution	KS: 5×5 Output: 33	Convolution	KS: 5×5 Output: 15	Convolution	KS: 3×3 Output: 27
Average Pooling	KS: 3×3 Strides: 2×2	Average Pooling	KS: 3×3 Strides: 2×2	Average Pooling	KS: 3×3 Strides: 2×2	Average Pooling	KS: 3×3 Strides: 2×2
Convolution	KS: 3×3 Output: 21	Convolution	KS: 3×3 Output: 21	Convolution	KS: 3×3 Output: 84	Convolution	KS: 4×4 Output: 41
Convolution	KS: 6×6 Output: 32	Convolution	KS: 5×5 Output: 23	Convolution	KS: 5×5 Output: 159	Fully-Connected	Output neurons: 10
Max Pooling	KS: 3×3 Strides: 2×2	Average Pooling	KS: 3×3 Strides: 2×2	Average Pooling	KS: 3×3 Strides: 2×2		
Convolution	KS: 5×5 Output: 35	Fully-Connected	Output neurons: 2	Fully-Connected	Output neurons: 10		
Fully-Connected	Output neurons: 10						

normalization in MNIST-Fashion dataset. Table 7 reports test errors and parms of methods in MNIST-Fashion dataset with and without dropout and batch normalization. Signs ‘+’ and ‘-’ in parentheses mean that the method has higher and lower test errors, respectively, in comparison with HeuCNN. Using these options, test error of HeuCNN is improved by 2.89 and 2.61 in “mean” and “best” cases, respectively. In these experiments, number of parameters is not decreased in networks. The reason is that using dropout and batch normalization leads to more complicated networks and increase the number of parameters. The best improvement of 16.69, 11.86, 11.39 and 10.29 were achieved in Human performance, MLP, GRU + SVM and AlexNet, respectively. Networks found by HeuCNN is very simpler architectures than other method (compare 62.3M parms in AlexNet than 116 K in HeuCNN).

4.7. Comparison with ViT models

Transformers have been used in many applications due to the success of vision transformer (ViT) models. In this section, 5 ViT models including: 1-Transformers for image recognition at scale referred as ViT-Base [43], 2-Hierarchical vision transformer using shifted Windows as swin-T [44], 3-Tokens-to-token vision transformer as T2T-ViT [45], 4-Convolutions to vision transformers as CvT [46] and 5-Efficient training of visual transformers with small-size datasets as CvT-drloc [47] are compared with the proposed model in 4 datasets MNIST, Fashion-MNIST, CIFAR-10 and CIFAR-100. Test errors of mentioned models and the proposed model are reported in Table 8. In MNIST dataset, since the test error of some superior models is less than 1%, the proposed method outperforms ViT models slightly by an improvement of 0.31% while in CIFAR-100 dataset accuracy is increased by 5.06%. The best cases of the proposed method in Fashion-MNIST and CIFAR-10 datasets outperforms the best ViT model CvT-drloc by 2.13% and 2.73%,

Table 7

Test errors and params of methods in MNIST-Fashion dataset with and without dropout and batch normalization.

Model	Test error	Params
Human performance	16.69%(+)	–
MLP 256-128-100	11.86%(+)	3M
GRU + SVM	11.39%(+)	–
HOG + SVM	7.59%(+)	–
AlexNet	10.29%(+)	62.3 M
3CONV + 3FC	6.79%(+)	500 k
VGG16	6.69%(+)	26 M
GoogLeNet	6.49%(+)	23 M
evoCNN (best)	5.28%(-)	6.68 M
evoCNN (mean)	7.14%(+)	6.52 M
MobileNet	4.81%(-)	4M
psoCNN (best)	0.19%(+)	2.32M
psoCNN (mean)	0.14%(-)	2.5M
HeuCNN-dropout-BN (best)	2.61%(+)	260.8 K
HeuCNN-dropout-BN (mean)	2.89%(+)	116.1 K
HeuCNN+dropout+BN (best)	5.34%	556.6 K
HeuCNN+dropout+BN (mean)	6.04%	661.5 K

Table 8

Test error of ViT models and the proposed model.

Method	MNIST	MNIST-Fashion	CIFAR-10	CIFAR-100
ViT-Base	16.12%	15.35%	22.52%	52.35%
Swin-T	1.89%	11.00%	20.43%	36.12%
T2T-ViT	1.59%	7.48%	13.88%	29.96%
CvT	0.98%	5.35%	7.98%	24.68%
CvT-drloc	0.68%	4.74%	5.70%	22.79%
Proposed (best)	0.37%	2.61%	2.97%	17.73%
Proposed (mean)	0.40%	2.89%	3.18%	19.13%

respectively. It means that the proposed method is the best method in all datasets in both best and average cases. The reason is that the proposed HeuCNN model finds the best layer structure and parameters for each dataset independently.

5. Discussion

Number of epochs for CNNs particles evaluation and weighting of objectives in HeuCNN are further discussed in this section.

5.1. Why PSO?

Each CNN architecture is represented as a particle in optimization space. In our model, CNN architectures are constructed with 4 types of layers including convolution, max pooling, average pooling and fully-connected layers. Number and order of layers are not same in different architectures. Therefore, variable-length optimization models should be used in this situation. PSO has more flexibility among variable-length swarm optimization models. Furthermore, the proposed model for CNN particles in multi-objective optimization space in Fig. 3 is more compatible with PSO since particles in PSO move in a 2-D optimization space with respect to two optimization criteria. All mathematical analysis for uniform and nonuniform distribution of particle, analyzing particle behaviors and operations such as difference and speed computation of CNN architectures are very close to basic concepts used in PSO. Therefore, PSO has been selected as a basic module of our optimization framework.

5.2. One epoch evaluation

Particle evaluation in optimization process is very important and it is a challenging task. Similar with many feature selection models that the whole classification process should be done to evaluate each solution in optimization space, CNN particles; as solution in optimization space; are evaluated by CNN training and test. Therefore, high number of CNN evaluations is needed. To address this issue, primary evaluation of CNNs is done with low number of epochs during optimization process and the final optimized CNN is trained with high number of epochs. For this task, CNNs are trained with 1 epoch (20 iterations). Table 9 reports combination of test error and params for 10 independent runs in Convex dataset. Two measures are combined and illustrated in Fig. 9. These experiments reveals that CNN cost function (which is a combination of two objectives) has downward trend in subsequent iterations although there are fluctuations in some iterations. Therefore, one-epoch execution can be used for CNN particles evaluation in optimization process.

Table 9
One epoch evaluation of CNN particles.

Iteration	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Run 7	Run 8	Run 9	Run 10
1	0.8769	1.0000	1.0000	0.6246	1.0000	0.5791	1.0000	1.0000	1.0000	0.9881
2	1.0000	0.9821	0.7694	0.5894	0.5731	0.3818	0.7441	0.9799	0.6454	0.6919
3	0.6742	0.9273	0.7407	0.9160	0.4982	0.9487	0.8066	0.8808	0.4900	0.6237
4	0.7197	0.7657	0.6625	1.0000	0.7263	0.4345	0.6192	0.7957	0.4836	0.6751
5	0.6723	0.4802	0.5631	0.8196	0.4787	0.4727	0.5989	0.8335	0.4402	0.5954
6	0.5739	0.5077	0.6179	0.6905	0.3894	0.4205	0.4941	0.5453	0.4667	0.4791
7	0.6629	0.4373	0.5379	0.4026	0.3713	0.3731	0.5259	0.5395	0.3989	0.4853
8	0.4394	0.4050	0.4420	0.4310	0.3368	0.3374	0.5266	0.4349	0.3545	0.4215
9	0.5095	0.4387	0.4160	0.4133	0.4055	0.3051	0.4785	0.4461	0.3650	0.4375
10	0.4943	0.3408	0.3887	0.3752	0.4184	0.3114	0.4672	0.3797	0.4172	0.4120
11	0.4943	0.3504	0.4147	0.3721	0.3691	0.2917	0.3980	0.2957	0.3300	0.3791
12	0.5057	0.3503	0.3959	0.3527	0.4263	0.2469	0.3631	0.3173	0.3498	0.3384
13	0.4621	0.3238	0.3629	0.3701	0.4020	0.2473	0.4180	0.3177	0.3634	0.2981
14	0.4318	0.3520	0.3139	0.3323	0.3986	0.2199	0.3415	0.2953	0.3130	0.3595
15	0.4186	0.2828	0.3507	0.3711	0.3429	0.2450	0.3419	0.4122	0.3136	0.2994
16	0.4508	0.3347	0.3534	0.3868	0.3591	0.2699	0.3633	0.4686	0.3287	0.2775
17	0.4621	0.2792	0.4034	0.4075	0.3612	0.1931	0.4220	0.3555	0.2955	0.2936
18	0.4394	0.3211	0.3373	0.3297	0.3425	0.2311	0.3627	0.3487	0.3294	0.2814
19	0.4186	0.2531	0.3320	0.3102	0.2887	0.2187	0.3216	0.2921	0.4005	0.2761
20	0.4091	0.2339	0.3186	0.3472	0.3240	0.1930	0.3413	0.3100	0.2778	0.2744

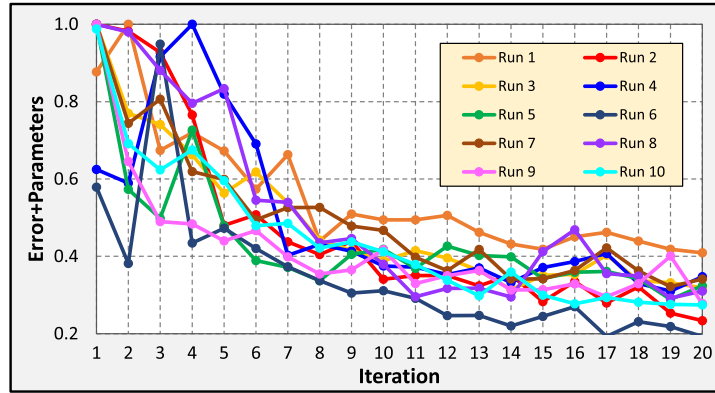


Fig. 9. Combination of test error and parms in one epoch evaluation.

5.3. Objectives weighting

Two objectives test error and parms are considered with the same weight (importance). These objectives can be optimized with different weights. It means that it is possible to force CNN particles to have better test error or better parms. In the case of “same weighting” which was used in all experiments, these objectives are optimized simultaneously. Different weights α_1 and α_2 for test error and parms, respectively, can be considered for dominant distance computation as $d_{D,mn} = \alpha_1 |E_m - E_n| + \alpha_2 |F_m - F_n|$. This measure can be extended for all particles in dominated region:

$$\begin{aligned}
 d_{D,m} &= \sum_{n=1}^K f(E_m, E_n, F_m, F_n) \cdot d_{D,mn} \\
 &= \sum_{n=1}^K f(E_m, E_n, F_m, F_n) \cdot (\alpha_1 |E_m - E_n| + \alpha_2 |F_m - F_n|)
 \end{aligned} \tag{33}$$

By the same way, distance of non-dominant particle can be defined as $d_{ND,mi} = \alpha_1 |E_m - E_i| + \alpha_2 |F_m - F_i|$ and for all particles in non-dominated region can be updated as:

$$\begin{aligned}
 d_{ND,m} &= \sum_{i=1}^K g(E_m, E_i, F_m, F_i) \cdot d_{ND,mi} \\
 &= \sum_{i=1}^K g(E_m, E_i, F_m, F_i) \cdot (\alpha_1 |E_m - E_i| + \alpha_2 |F_m - F_i|)
 \end{aligned} \tag{34}$$

Table 10
Applying different values of α_1 and α_2 in Convex dataset.

weight parameter	weight error	Parameter	Error
0.1	0.9	13546	6.47
0.2	0.8	47543	5.76
0.3	0.7	65185	5.47
0.4	0.6	58974	4.71
0.5	0.5	98765	2.56
0.6	0.4	764987	2.19
0.7	0.3	986376	1.88
0.8	0.2	865397	1.92
0.9	0.1	1423454	1.95

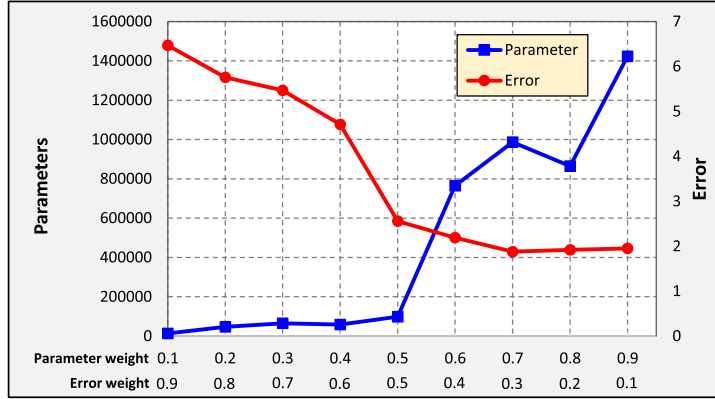


Fig. 10. Test errors and parms for different weights of objective in optimization space.

Table 10 reports test error and parms of HeuCNN with different pairs (0.1,0.9), (0.2,0.8), (0.3,0.7), (0.4,0.6), (0.5,0.5), (0.6,0.4), (0.7,0.3), (0.8,0.2) and (0.9,0.1) for (α_1, α_2) . These objectives are also illustrated in Fig. 10. The main conclusion from these experiments is that assigning higher weights to an objective leads to better performance in that objective and weaker performance in another. For example, pairs (0.1,0.9) results in 13k parms with test error 6.47 while (0.9,0.1) makes an optimized CNN with 1423k parms and 1.95 of test error in Convex dataset.

6. Limitation of HeuCNN as a future work

Order and type of layers make different CNN architectures. Optimization methods generate different combinations of layers and evaluate each one to find the best architecture. Each layer has its own hyper-parameters including filter size, number of filters, stride and zero-padding which affect CNN performance significantly. The proposed HeuCNN and all existing optimization models use a constant interval for hyper-parameters of all layers. For example, dimension of kernels in all convolution layers is randomly selected from interval $[min_{KD}, max_{KD}]$. This interval should be configured carefully since it should be applied to all layers. For i th layer in a CNN model, dimension of input image, position of this layer (its distance from input and output layers) and dimension of image reaching to this layer affect interval $[min_{KD}, max_{KD}]$. It means that intervals should be computed in adaptive manner for each layer. It is worth mentioned that considering constant intervals for hyper-parameters of all CNN layers limits optimization methods to explore optimization space. This is the main limitation of HeuCNN. To address this limitation, the following idea is proposed as a future work. Suppose a conventional CNN architecture including NB blocks, M convolution layer and 1 pooling layer per block. Passing input image I_0 from one convolution layer leads to the output $I_{out} = I_0 + Z_{i,k} - C_{i,k} + 1$, where $C_{i,k}$ and $Z_{i,k}$ are convolution kernel dimension and zero-padding in k th layer of i th block. Applying 1 convolution layer and 1 pooling layer to input image leads to the following output:

$$I_{out} = \left\lfloor \frac{I_0 + Z_{i,k} - C_{i,k} + 1 - P_{i,k} + S_{i,k}}{S_{i,k}} \right\rfloor \quad (35)$$

where $P_{i,k}$ and $S_{i,k}$ are respectively pooling kernel dimension and stride in k th layer of i th block. This equation can be extended to all layers which make the output dimension I_{NB} in the final block as:

$$I_{NB} \geq \frac{I_0 + \sum_{j=1}^M \sum_{k=1}^{NB} \left[\left(\prod_{m=1}^k S_{m-1} \right) (Z_{k,j} - C_{k,j}) \right] + \sum_{k=1}^{NB} \left[\left(\prod_{m=1}^k S_{m-1} \right) (M - P_k) \right]}{\prod_{k=1}^{NB} S_k} \quad (36)$$

The content of the last block should be non-empty ($I_{NB} > 1$). Based on this constraint and Eq. (36), the following constraint should be satisfied for the i th block:

$$I_i \geq \underbrace{\prod_{m=i+1}^{NB} S_m - \sum_{j=1}^M \sum_{k=i+1}^{NB} \left[\left(\prod_{m=i+2}^k S_{m-1} \right) (Z_{k,j} - C_{k,j}) \right]}_{R_i} - \sum_{k=i+1}^{NB} \left[\left(\prod_{m=i+2}^k S_{m-1} \right) (NB - P_k) \right] \quad (37)$$

where I_i is the output dimension in i th block. Satisfying this constraint results in computing adaptive interval for hyper-parameters such as $C_{i,k}$:

$$C_{i,k} \in \left[C_{min}, I_{i-1} + \sum_{m=1}^k Z_{i,m} + (M - k) (Z_{max} - C_{min}) - \sum_{m=1}^{k-1} C_{i,m} + M - P_{min} - R_i S_{min} \right] \quad (38)$$

By the same way, other constraints are applicable to hyper-parameters and adaptive intervals can be computed in each CNN layer.

7. Conclusion

Convolutional neural networks (CNNs) are among the most significant deep learning models for image classification. Due to the difficulties in configuration of layers and parameters in CNNs, a heuristic-based architecture optimization model (HeuCNN) was presented in this research. For this task, architecture and parameters of CNNs were encoded as PSO particles and evaluated by the proposed heuristic in optimization space. Furthermore, comprehensive mathematical and experimental analysis were done for uniform and nonuniform distribution of CNN particles. Then, new rules for particle difference computation, particle velocity computation and particle update were introduced which led to PSO-based architecture optimization. Experiments showed that HeuCNN has promising classification performance in comparison with state-of-the-art CNN models. As mentioned in details, computing adaptive intervals for parameters of each CNN layer was recommended to address the weakness of HeuCNN and can be proposed as a future work. Also, CNNs can be considered as compression tools since they pass high-dimensional inputs from convolution and pooling layers and connect them to FC layers. Therefore, future efforts can be directed towards optimization of down-sampling part of auto-encoder structures which are similar to CNN models. Finally, in optimization process of CNN, parameters are selected randomly from a constant interval. Future efforts will also be directed towards automatic determination intervals based on input dimension in each layer.

CRedit authorship contribution statement

Armin Rashno: Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Resources, Investigation, Data curation. **Sadegh Fadaei:** Visualization, Validation, Software, Methodology, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] Z. Li, F. Liu, W. Yang, S. Peng, J. Zhou, A survey of convolutional neural networks: analysis, applications, and prospects, *IEEE Trans. Neural Netw. Learn. Syst.* 33 (12) (2022) 6999–7019.
- [2] A. Khan, A. Sohail, U. Zahoor, A.S. Qureshi, A survey of the recent architectures of deep convolutional neural networks, *Artif. Intell. Rev.* 53 (8) (2020) 5455–5516.
- [3] Y. Zhang, Y. Liu, X.S. Liu, Neural network architecture search with amber, *Nat. Mach. Intell.* 3 (5) (2021) 372–373.
- [4] K.O. Stanley, R. Miikkulainen, Evolving neural networks through augmenting topologies, *Evol. Comput.* 10 (2) (2002) 99–127.
- [5] A. Rashno, S. Sadri, H. SadeghianNejad, An efficient content-based image retrieval with ant colony optimization feature selection schema based on wavelet and color features, in: 2015 the International Symposium on Artificial Intelligence and Signal Processing (AISP), IEEE, 2015, pp. 59–64.
- [6] A. Rashno, B. Nazari, S. Sadri, M. Saraee, Effective pixel classification of Mars images based on ant colony optimization feature selection and extreme learning machine, *Neurocomputing* 226 (2017) 66–79.
- [7] A. Rashno, S.M. Ahadi, M. Kelarestaghi, Text-independent speaker verification with ant colony optimization feature selection and support vector machine, in: 2015 2nd International Conference on Pattern Recognition and Image Analysis (IPRIA), IEEE, 2015, pp. 1–5.
- [8] M. Shafipour, A. Rashno, S. Fadaei, Particle distance rank feature selection by particle swarm optimization, *Expert Syst. Appl.* 185 (2021) 115620.
- [9] A. Rashno, M. Shafipour, S. Fadaei, Particle ranking: an efficient method for multi-objective particle swarm optimization feature selection, *Knowl.-Based Syst.* 245 (2022) 108640.
- [10] F. Valdez, J.C. Vazquez, P. Melin, O. Castillo, Comparative study of the use of fuzzy logic in improving particle swarm optimization variants for mathematical functions using co-evolution, *Appl. Soft Comput.* 52 (2017) 1070–1083.

- [11] M.K. Chegeni, A. Rashno, S. Fadaei, Convolution-layer parameters optimization in convolutional neural networks, *Knowl.-Based Syst.* 261 (2023) 110210.
- [12] A. Rashno, D.D. Koozekanani, K.K. Parhi, Oct fluid segmentation using graph shortest path and convolutional neural network, in: 2018 40th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), IEEE, 2018, pp. 3426–3429.
- [13] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, *Proc. IEEE* 86 (11) (1998) 2278–2324.
- [14] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Adv. Neural Inf. Process. Syst.* 25 (2012) 1097–1105.
- [15] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, *arXiv preprint*, arXiv:1409.1556.
- [16] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 1–9.
- [17] J. Bruna, S. Mallat, Invariant scattering convolution networks, *IEEE Trans. Pattern Anal. Mach. Intell.* 35 (8) (2013) 1872–1886.
- [18] H. Larochelle, D. Erhan, A. Courville, J. Bergstra, Y. Bengio, An empirical evaluation of deep architectures on problems with many factors of variation, in: *Proceedings of the 24th International Conference on Machine Learning*, 2007, pp. 473–480.
- [19] T.-H. Chan, K. Jia, S. Gao, J. Lu, Z. Zeng, Y. Ma, Pcanet: a simple deep learning baseline for image classification?, *IEEE Trans. Image Process.* 24 (12) (2015) 5017–5032.
- [20] K. Sohn, G. Zhou, C. Lee, H. Lee, Learning and selecting features jointly with point-wise gated Boltzmann machines, in: *International Conference on Machine Learning*, PMLR, 2013, pp. 217–225.
- [21] K. Sohn, H. Lee, Learning invariant representations with local transformations, *arXiv preprint*, arXiv:1206.6418.
- [22] L. Wan, M. Zeiler, S. Zhang, Y. Le Cun, R. Fergus, Regularization of neural networks using dropconnect, in: *International Conference on Machine Learning*, PMLR, 2013, pp. 1058–1066.
- [23] S. Rifai, P. Vincent, X. Muller, X. Glorot, Y. Bengio, Contractive auto-encoders: explicit invariance during feature extraction, in: *ICML*, 2011.
- [24] C. Sui, M. Bennamoun, R. Togneri, Deep feature learning for dummies: a simple auto-encoder training method using particle swarm optimisation, *Pattern Recognit. Lett.* 94 (2017) 75–80.
- [25] Y. Sun, B. Xue, M. Zhang, G.G. Yen, A particle swarm optimization-based flexible convolutional autoencoder for image classification, *IEEE Trans. Neural Netw. Learn. Syst.* 30 (8) (2018) 2295–2309.
- [26] I. Strumberger, E. Tuba, N. Bacanin, M. Zivkovic, M. Beko, M. Tuba, Designing convolutional neural network architecture by the firefly algorithm, in: 2019 International Young Engineers Forum (YEF-ECE), IEEE, 2019, pp. 59–65.
- [27] I. Strumberger, E. Tuba, N. Bacanin, R. Jovanovic, M. Tuba, Convolutional neural network architecture design by the tree growth algorithm framework, in: 2019 International Joint Conference on Neural Networks (IJCNN), IEEE, 2019, pp. 1–8.
- [28] B. Wang, Y. Sun, B. Xue, M. Zhang, Evolving deep convolutional neural networks by variable-length particle swarm optimization for image classification, in: 2018 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2018, pp. 1–8.
- [29] Y. Li, J. Xiao, Y. Chen, L. Jiao, Evolving deep convolutional neural networks by quantum behaved particle swarm optimization with binary encoding for image classification, *Neurocomputing* 362 (2019) 156–165.
- [30] F.E.F. Junior, G.G. Yen, Particle swarm optimization of deep neural networks architectures for image classification, *Swarm Evol. Comput.* 49 (2019) 62–74.
- [31] Y. Wang, H. Zhang, G. Zhang, cpso-cnn: an efficient pso-based algorithm for fine-tuning hyper-parameters of convolutional neural networks, *Swarm Evol. Comput.* 49 (2019) 114–123.
- [32] N. Bacanin, T. Bezdan, E. Tuba, I. Strumberger, M. Tuba, Optimizing convolutional neural network hyperparameters by enhanced swarm intelligence metaheuristics, *Algorithms* 13 (3) (2020) 67.
- [33] P. Singh, S. Chaudhury, B.K. Panigrahi, Hybrid mpso-cnn: multi-level particle swarm optimized hyperparameters of convolutional neural network, *Swarm Evol. Comput.* 63 (2021) 100863.
- [34] G.H. De Rosa, J.P. Papa, X.-S. Yang, Handling dropout probability estimation in convolution neural networks using meta-heuristics, *Soft Comput.* 22 (18) (2018) 6147–6156.
- [35] Y. Sun, B. Xue, M. Zhang, G.G. Yen, Evolving deep convolutional neural networks for image classification, *IEEE Trans. Evol. Comput.* 24 (2) (2019) 394–407.
- [36] J. Jiang, F. Han, Q. Ling, J. Wang, T. Li, H. Han, Efficient network architecture search via multiobjective particle swarm optimization based on decomposition, *Neural Netw.* 123 (2020) 305–316.
- [37] E. Bochinski, T. Senst, T. Sikora, Hyper-parameter optimization for convolutional neural network committees based on evolutionary algorithms, in: 2017 IEEE International Conference on Image Processing (ICIP), IEEE, 2017, pp. 3924–3928.
- [38] Y. Sun, G.G. Yen, Z. Yi, Evolving unsupervised deep neural networks for learning meaningful representations, *IEEE Trans. Evol. Comput.* 23 (1) (2018) 89–103.
- [39] T. Yamasaki, T. Honma, K. Aizawa, Efficient optimization of convolutional neural networks using particle swarm optimization, in: 2017 IEEE Third International Conference on Multimedia Big Data (BigMM), IEEE, 2017, pp. 70–73.
- [40] T. Sinha, B. Verma, A. Haidar, Optimization of convolutional neural network parameters for image classification, in: 2017 IEEE Symposium Series on Computational Intelligence (SSCI), IEEE, 2017, pp. 1–7.
- [41] N.M. Aszemi, P. Dominic, Hyperparameter optimization in convolutional neural network using genetic algorithms, *Int. J. Adv. Comput. Sci. Appl.* 10 (6) (2019) 269–278.
- [42] H. Louati, S. Bechikh, A. Louati, C.-C. Hung, L.B. Said, Deep convolutional neural network architecture design as a bi-level optimization problem, *Neurocomputing* 439 (2021) 44–62.
- [43] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., An image is worth 16x16 words: transformers for image recognition at scale, *arXiv preprint*, arXiv:2010.11929.
- [44] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, B. Guo, Swin transformer: hierarchical vision transformer using shifted windows, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 10012–10022.
- [45] L. Yuan, Y. Chen, T. Wang, W. Yu, Y. Shi, Z.-H. Jiang, F.E. Tay, J. Feng, S. Yan, Tokens-to-token ViT: training vision transformers from scratch on ImageNet, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 558–567.
- [46] H. Wu, B. Xiao, N. Codella, M. Liu, X. Dai, L. Yuan, L. Zhang, Cvt: introducing convolutions to vision transformers, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 22–31.
- [47] Y. Liu, E. Sangineto, W. Bi, N. Sebe, B. Lepri, M. Nadai, Efficient training of visual transformers with small datasets, *Adv. Neural Inf. Process. Syst.* 34 (2021) 23818–23830.